计 算 机 体 系 结 构

第十讲

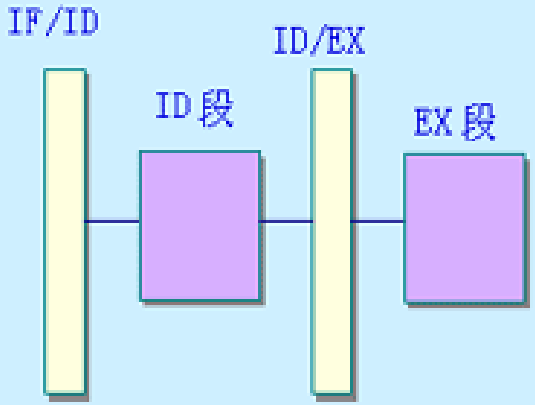
计算机科学与技术学院
舒燕君

Recap

- 流水线数据冲突
  - ✓ 数据冲突的分类
  - ✓ 需要暂停的数据冲突
  - ✓ 通过编译调度解决的数据冲突
  - ✓ 流水线锁的检测与实现
  - ✓ 定向逻辑的实现
- 流水线控制冲突
  - ✓减少分支开销的途径（改进、再改进）
  - ✓减少流水线分支损失的方法

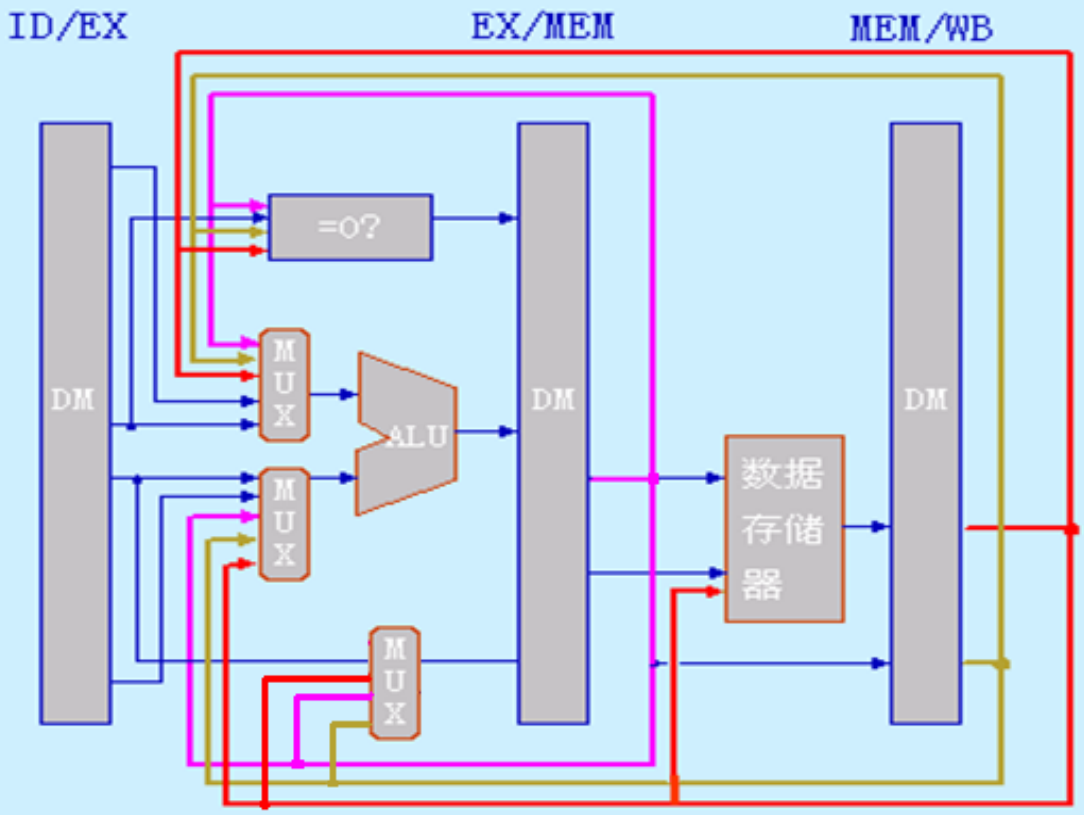
在ID段检测Load互锁需进行三种比较

- Load、Store、ALU 立即数、分支等指令：
  - 源寄存器rs1：$IR_{6...10}$
  - 结果寄存器rd：$IR_{11...15}$
- R-R ALU指令：
  - 源寄存器rs1：$IR_{6...10}$
  - 源寄存器rs2：$IR_{11...15}$
  - 结果寄存器rd：$IR_{16...20}$



ID/EX 的操作码域 (ID/EX.$IR_{0...5}$)	IF/ID的操作码域 (IF/ID.$IR_{0...5}$)	匹配操作数域
Load	R-R ALU	ID/EX.$IR_{11...15}$ =IF/ID.$IR_{6...10}$
Load	R-R ALU	ID/EX.$IR_{11...15}$ =IF/ID.$IR_{11...15}$
Load	Load、Store、ALU立即数或分支	ID/EX.$IR_{11...15}$ =IF/ID.$IR_{6...10}$

定 向 路 径



3.3.3 流水线的控制冲突

1. 分支指令的实现

- 一旦分支转移成功，正确的地址要在Mem段才会被写入PC。
- 一旦ID段检测到分支指令，就暂停执行其后的指令，直到分支指令达到Mem段，确定新的PC为止。
- 分支转移成功将导致MIPS流水线暂停3个周期。

3.3.3 流水线的控制冲突

2. 减少分支开销的途径

- 两个基本途径：同时采用，缺一不可！
  - ✓ 在流水线中尽早判断分支转移是否成功
  - ✓ 转移成功时，尽早计算出转移目标地址

- 经改进，MIPS流水线可以将分支开销减少1拍
  - ✓ 将“=0?”测试提前到ID段
  - ✓ 在ID段增加一个加法器，计算分支目标地址

  下表列出了改进后流水线的分支操作

- 再改进，MIPS流水线可以将分支开销再减少1拍
  - ✓ 将分支判断结果和目标地址提前到ID/EX站前

3.3.3 流水线的控制冲突

3. 程序中分支指令的行为特点

(1) 各种能改变PC值的指令的执行频度

- 条件分支：
  - 整数程序：14-15%
  - 浮点程序：3-12%

  其中，向前分支与向后分支的比：3:1
- 无条件分支：≤4%（绝大多数）

(2) 条件分支转移成功的概率

- 向前转移成功率：60%；向后转移成功率：85%

3.3.3 流水线的控制冲突

4. 减少流水线分支损失的方法

(1) 冻结或排空流水线

- 思路：在流水线中停住或删除分支后的指令，直到知道转移目标地址
- 优点：简单

(2) 预测分支转移失败

- 思路：流水线继续照常流动，如果分支转移成功，将分支指令后的指令转换为空操作，并从分支目标处开始取指令执行；否则照常执行
- MIPS流水线的处理过程

3.3.3 流水线的控制冲突

4. 减少流水线分支损失的方法

(3) 预测分支转移成功

- 思路：始终假设分支成功，直接从分支目标处取指令执行
- 对MIPS流水线没有任何好处！

(4) 延迟分支（delayed branch）

- 思路：分支开销为n的分支指令后紧跟有n个延迟槽，流水线遇到分支指令时，按正常方式处理，顺带执行延迟槽中的指令，从而减少分支开销。

  延迟分支及指令的执行顺序

3.3.3 流水线的控制冲突

4. 减少流水线分支损失的方法

（1）具有一个分支延迟槽的MIPS流水线的执行过程

（2）什么样的指令能否放入分支延迟槽？

- 三种调度方法：从前调度；从目标处调度；从失败处调度
- 三种方法的要求与效果，存在限制因素
  - ✓ 编译器预测分支是否成功的能力
  - ✓ 放入延迟槽中的指令
- 取消分支　　（举例）

  思路：分支指令中包含预测方向，若预测正确，正常执行延迟槽中的指令，否则将其转换为空操作

3.3.3 流水线的控制冲突

5. 各种分支处理方法的性能

(1) 假设理想**CPI=1**，则加速比

$$S=D/(1+C)=D/(1+f\times p_{分支})$$

这里，**D**为流水线的深度，$p_{分支}$为分支开销，**C**为分支引起的流水线暂停时钟周期数(每条指令的平均值)，**f**为分支的出现频度。

(2) 下表列出了流水线中各种处理方法的开销。

各种减少分支损失方法的效果

调度方法	每条条件分支指令的分支损失		每条无条件分支指令的损失	每条分支指令的平均分支损失		具有分支暂停的有效 CPI	
	整型平均	浮点平均		整型平均	浮点平均	整型平均	浮点平均
暂停流水线	1.00	1.00	1.00	1.00	1.00	1.17	1.15
预测分支成功	1.00	1.00	1.00	1.00	1.00	1.17	1.15
预测分支失败	0.62	0.70	1.00	0.69	0.74	1.12	1.11
延迟分支	0.25	0.35	0.00	0.21	0.30	1.04	1.04

3.4 实例分析：MIPS R4000

3.4.1 MIPS R4000的整型流水线

3.4.2 MIPS R4000的浮点流水线

3.4.3 MIPS R4000流水线性能分析

3.4.1 MIPS R4000的整型流水线

1. 指令集：64位MIPS-3指令集

2. MIPS R4000流水线结构

   - 超流水结构（Superpipeling）
   - 访存操作流水化

3. 流水线各段的功能

3.4.1 MIPS R4000的整型流水线

4. 指令序列在流水线中的重叠执行过程

定向+插入暂停周期

5. 载入延迟为两个时钟周期

6. 指令序列在流水线中的执行时空图

7. R4000流水线的定向路径远多于MIPS流水线

ALU输入端的定向源有4个：EX/DF，DF/DS，DS/TC，TC/WB

3.4.1 MIPS R4000的整型流水线

8. 分支处理

- 在EX段完成分支条件的计算，基本分支延迟为3个时钟周期

- 分支处理策略
  - 单周期延迟分支
  - 从失败处调度　　时-空图

3.4.2 MIPS R4000的浮点流水线

- MIPS R4000 FPU包括浮点除法器、浮点乘法器和浮点加法器各1个
- 分为8段（R4000流水线 FPU的8个段）
- 多功能非线性流水线
- 双精度浮点操作指令延迟、初始化间隔和流水段的使用情况（使用情况）

3.4.3 MIPS R4000流水线性能分析

1. 引起流水线暂停的四个主要原因
   - 载入暂停
   - 分支暂停
   - 浮点结果暂停
   - 浮点结构性暂停

2. 暂停对MIPS R4000流水线CPI的影响

暂停对R4000流水线CPI的影响

整数平均:	流水线CPI	1.54
	载入暂停时钟周期数	0.16
	分支暂停时钟周期数	0.38
	浮点结果暂停时钟周期数	0.00
	浮点结构性暂停时钟周期数	0.00
浮点平均:	流水线CPI	2.48
	载入暂停时钟周期数	0.10
	分支暂停时钟周期数	0.33
	浮点结果暂停时钟周期数	0.95
	浮点结构性暂停时钟周期数	0.18

3.5 向量处理机实例：Cray-I

- 什么是向量机？

  具有向量数据表示和相应向量指令的流水线处理机称为向量流水线处理机，也称向量处理机。

  与之对应的是标量处理机，不支持向量数据表示，没有提供向量指令。

- 一个简单的FORTRAN循环程序

```
        DO 10 i=1,N
10      d[i] = a[i]*(b[i]+c[i])
```

A、B、C、D四个向量

水平（横向）处理方式

- 依次计算向量k和d的每个元素

  … …

  $k_i = b_i + c_i$

  $d_i = k_i * a_i$

  分支 $i <= N$?

  … …

- 循环的每个迭代(iteration)中有1次数据相关，1次控制相关，需要2次功能切换

垂直（纵向）处理方式

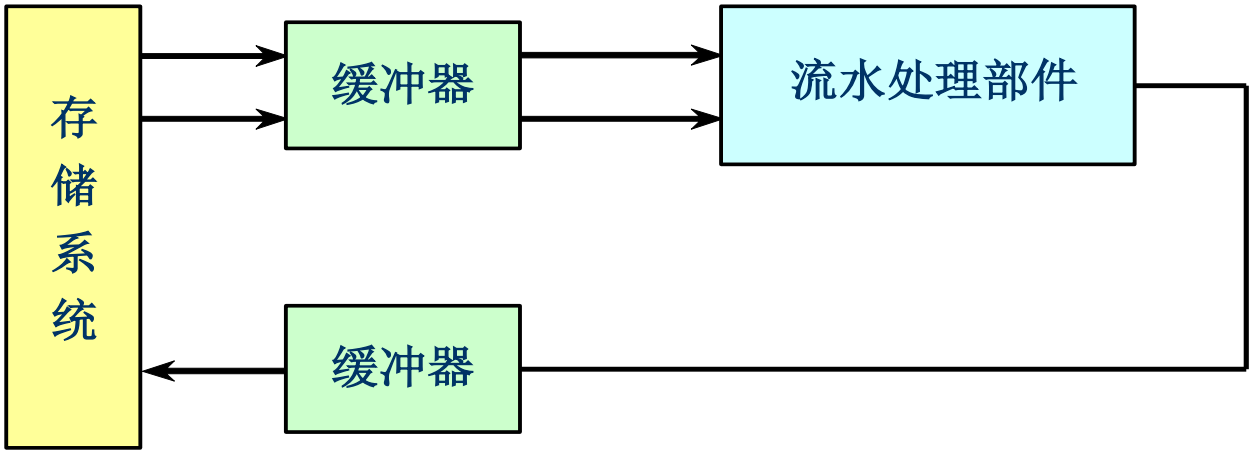
- 计算向量k的每个元素之后，再计算向量d的每个元素，需要向量数据类型和向量指令的支持

  K = B + C

  D = K * A

- 没有分支；仅有1次数据相关；仅需要1次功能切换

- 需要存储器-存储器型操作的运算流水线



分组（纵横）处理方式

- 将长度为N的向量分为m组，每组有n个元素，组内按纵向方式处理，依次处理各组。
- 需要m次迭代；每次迭代执行两条向量指令，有1次数据相关，需要2次功能切换
- 需要寄存器-寄存器型操作的运算流水线
- 这种技术称为向量循环或分段开采

例3.8 设A和B是长度为N的向量，考虑在一个向量长度为64的向量处理机上实现如下的循环操作：

```
      DO 10 I=1, N
10    A(I)=5.0 * B(I) + 1.0
```

分析：

(1) 当$N \leqslant 64$时

(2) 当$N>64$时，需要进行分段开采

4. 向量处理机的速度评价方法

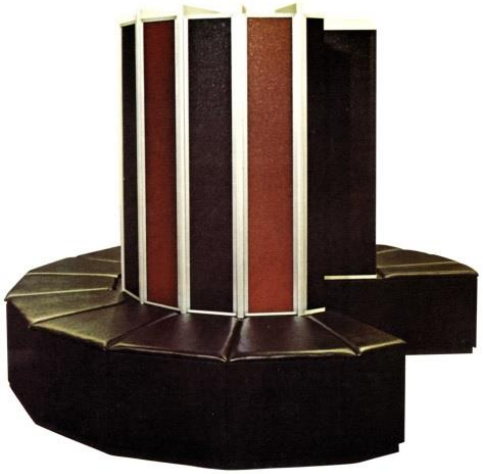
- 由于一条指令最多得到一个结果，标量处理机通常用每秒执行多少条指令(MIPS)来衡量机器的运算速度
- 向量处理机用每秒取得多少浮点运算结果来衡量机器速度，以MFLOPS作为测量单位
- 采用MFLOPS可以忽视Load、Store、分支、测试等类型指令的影响

4. 向量处理机的速度评价方法

- 一般认为，在标量计算机中执行一次浮点运算，平均需3条指令。因此，如果要把这两种速度指标放在一起比较，就应该把MFLOPS乘以一个系数，以得到相应的MIPS。

- 比较法：选择一台速度指标得到公认的机器作为标准机，给定一些典型的基准程序，分别在标准机和被测机上求解。

3.5 向量处理机实例：Cray-I



1. 性能指标

- **1GFLOPS**、主频**80M**、向量长度**64**
- 无**Data Cache**，不支持**Virtual Memory**

2. 基本结构

- 功能部件：**12**条并行工作的功能单元流水线，分别进行地址、向量、标量、浮点运算。
- 向量运算：整数加（**3**拍）、逻辑运算（**2**拍）、移位（**4**拍）、浮点加（**6**拍）、浮点乘（**7**拍）、浮点求倒数（**14**拍）
- 向量寄存器V_0-V_7
- 向量长度寄存器、向量屏蔽寄存器

向量处理机实例：Cray-I

3. 向量指令类型

- $V_k \leftarrow V_i$ op V_j
- $V_k \leftarrow S_i$ op V_j
- $V_k \leftarrow$ Mem
- Mem $\leftarrow V_k$

➢ 功能部件冲突：同一功能部件被一条以上的并行工作向量指令所使用。

➢ V_i冲突：并行工作的各向量指令具有相同的源向量或结果向量。

向量处理机实例：Cray-I

4. CRAY-I体系结构特点

- 向量寄存器与功能单元的连接通路

每个Vi块都有单独总线可连到6个向量功能部件，而每个向量功能部件也各自都有把运算结果送回向量寄存器组的总线。

- 向量链接技术

一个向量功能部件得到的结果直接送入另一个向量功能部件的操作数寄存器时所发生的连接过程称为链接（Chaining）。

当两条指令出现“先写后读（RAW）”相关时，若它们不存在功能部件冲突和向量寄存器(源或目的) 冲突，就有可能把它们所用的功能部件头尾相接，形成一个链接流水线，进行流水处理。

链接特性实质上是把流水线“定向”的思想引入到向量执行过程的结果。

5. 向量链接技术实例分析

例：对向量运算D=A*(B+C)，若向量长度N≤64，

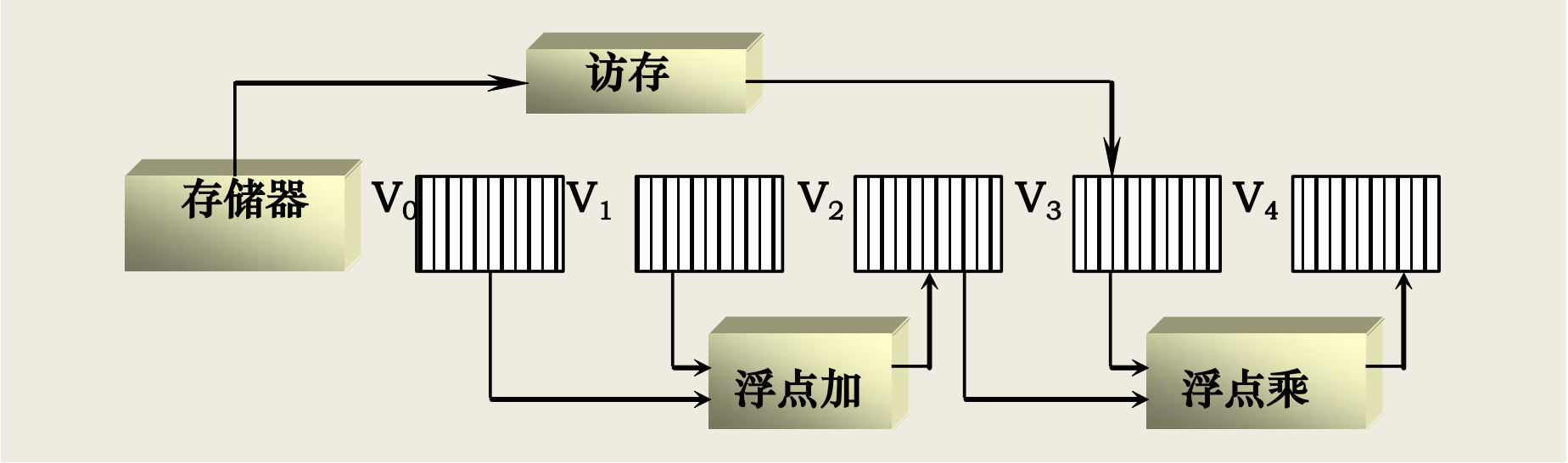
向量元素为浮点数，则在B、C取到V0、V1后，

就可用以下三条向量指令求解：

V3←存储器　(访存，载入A)

V2←V0＋V1　(浮点加)

V4←V2*V3　(浮点乘，将D存入V4)



假设：向量处理机将元素从Vi送往功能部件及把结果存入Vi都需要1拍；浮点加法和访存操作都需要6拍；浮点乘操作需要7拍。

- 3向量条指令全部串行：

  [(1+6+1)+N-1]+[(1+6+1)+N-1]+[(1+7+1)+N-1]=3N+22 拍

- 第1、2条向量指令并行，再串行执行第3条向量指令：

  [(1+6+1)+N-1]+[(1+7+1)+N-1]=2N+15 拍

- 第1、2条向量指令并行，再第3条向量指令链接执行：

  第一个结果被存入V4需要经过（链接流水线建立时间）：

  1(送)+ 6(浮加) +1(入)+1(送)+7(浮乘)+1(入)=17(拍)

  此后，每拍将得到一个结果送入V4。

  总的完成时间为：17+（N-1）拍

6. 向量链接技术应考虑的问题

- 设定合适的向量功能部件和操作数寄存器
- 链接时机问题
  - 只有在前一条向量指令的第一个结果元素送入结果向量寄存器的那一个时钟周期才可以进行链接
  - 当一条向量指令的两个源操作数分别是两条先行指令的结果寄存器时，要求先行的两条指令产生运算结果的时间必须相等，即要求有关功能部件的通过时间相等。
  - 所有可以链接执行的向量指令的向量长度应相等

向量机总结

- 向量机适合挖据规整的数据级并行
  - 同样的操作作用在许多数据元素上
  - 提高性能、设计简单（向量内的操作相互独立）
- 性能的提升受限于代码的向量化
  - 标量操作限制着向量机的性能
- 很多已有的ISA扩展了一些SIMD操作
  - **Intel MMX/SSEn/AVX, PowerPC AltiVec, ARM Advanced SIMD**
- 另一种SIMD变体，图形处理单元GPU

Graphics Processing Units (GPUs)

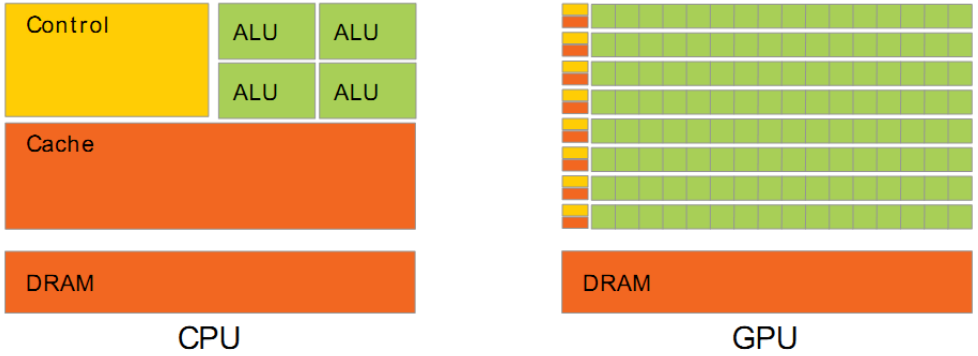
- 早期的GPU是指带有高性能浮点运算部件、可高效生成3D图形的具有固定功能的专用设备 (mid-late 1990s)
  - 让PC机具有类似工作站的图形功能
  - 用户可以配置图形处理流水线，但不是真正的对其编程
- 2001-2005， GPU加入了越来越多的可编程性
  - 例如新的语言 Cg可用来编写一些小的程序处理图形的顶点或像素，是Windows DirectX的变体
  - 大规模并行（针对每帧上百万顶点或像素）但非常受限于编程模型
- 通过将输入和输出数据映射为图像，并对顶点或像素渲染计算 可进行通用计算
  - 因为不得不使用图形流水线模型，这对完成通用计算来说是个非常难用的编程模型

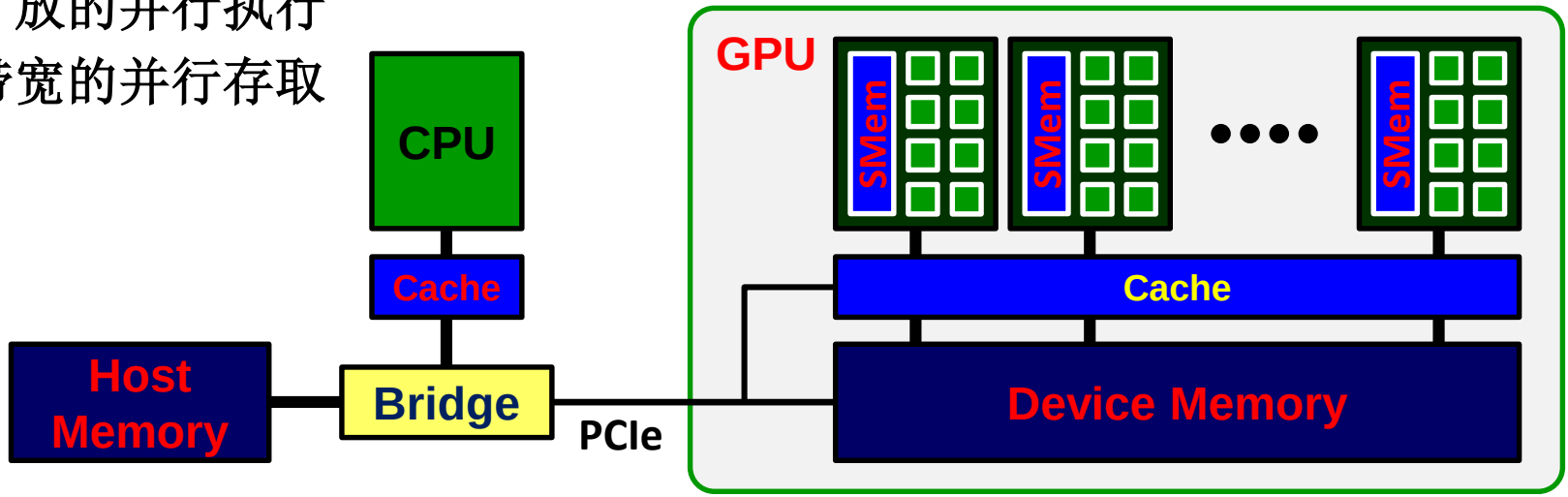
General-Purpose GPUs (GP-GPUs)

- **2006年, Nvidia 的 GeForce 8800 GPU 支持一种新的编程语言: CUDA**
  - **“Compute Unified Device Architecture”**
  - 随后工业界推出**OpenCL**，与**CUDA**具有相同的**ideas,** 但独立于供应商
- **Idea:** 针对通用计算，发挥**GPU**计算的高性能和存储器的高带宽来加速一些通用计算中的核心（**Kernels**）
- 一种附加处理器模型（**GPU**作为附加设备）：**Host CPU** 发射数据并行的**kernels** 到**GP-GPU**上运行
- 仅讨论**Nvidia CUDA**样式的简化版本，仅考虑**GPU**的计算核部分，不涉及图形加速部分

Using CPU+GPU Architecture

- **CPU+GPU异构多核系统**
  - 针对每个任务选择合适的处理器和存储器
- **通用CPU 适合执行一些串行的线程**
  - 串行执行快
  - 带有cache，访问存储器延时低
- **GPU 适合执行大量并行线程**
  - 可扩放的并行执行
  - 高带宽的并行存取





CPU与GPU的性能比较

	Num. PE	Frequency (GHz)	Num. Transistors	BW (GB/sec)	SP SIMD width	DP SIMD width	Peak SP Scalar FLOPS (GFLOPS)	Peak SP SIMD Flops (GFLOPS)	Peak DP SIMD Flops (GFLOPS)
Core i7-960	4	3.2	0.7B	32	4	2	25.6	102.4	51.2
GTX280	30	1.3	1.4B	141	8	1	116.6	311.1/933.1	77.8

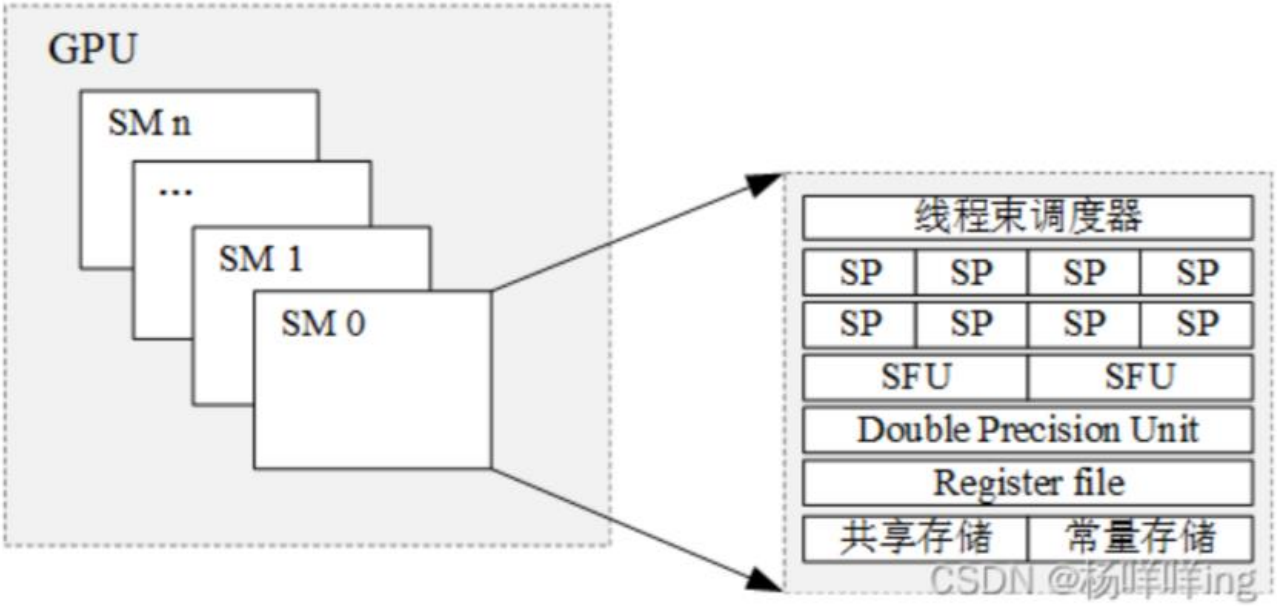
Table 2: Core i7 and GTX280 specifications. BW: local DRAM bandwidth, SP: Single-Precision Floating Point, DP: Double-Precision Floating Point.

Apps.	SGEMM	MC	Conv	FFT	SAXPY	LBM	Solv	SpMV	GJK	Sort	RC	Search	Hist	Bilat
Core i7-960	94	0.8	1250	71.4	16.8	85	103	4.9	67	250	5	50	1517	83
GTX280	364	1.4	3500	213	88.8	426	52	9.1	1020	198	8.1	90	2583	475

Table 3: Raw performance measured on the two platforms. From the left, metrics are Gflops/s, billion paths/s, million pixels/s, Gflops/s, GB/s, million lookups/s, FPS, Gflops/s, FPS, million elements/s, FPS, million queries/s, million pixels/s, million pixels/s.

CSDN @GutsShinyHero

- **SM（Streaming Multiprocessor）**
  - 每个包含8个流处理器（Streaming Processor， SP）及特殊函数单元（Special Function Unit，SFU）、寄存器、共享内存和常量缓存。
  - 在执行高密度计算时，每一个SM都可以实现数以千计个线程的并发执行，当核函数被启动时，GPU会根据分配策略将线程块分配到相应的SM执行，但是不同架构下单个SM的常驻线程数也不尽相同。



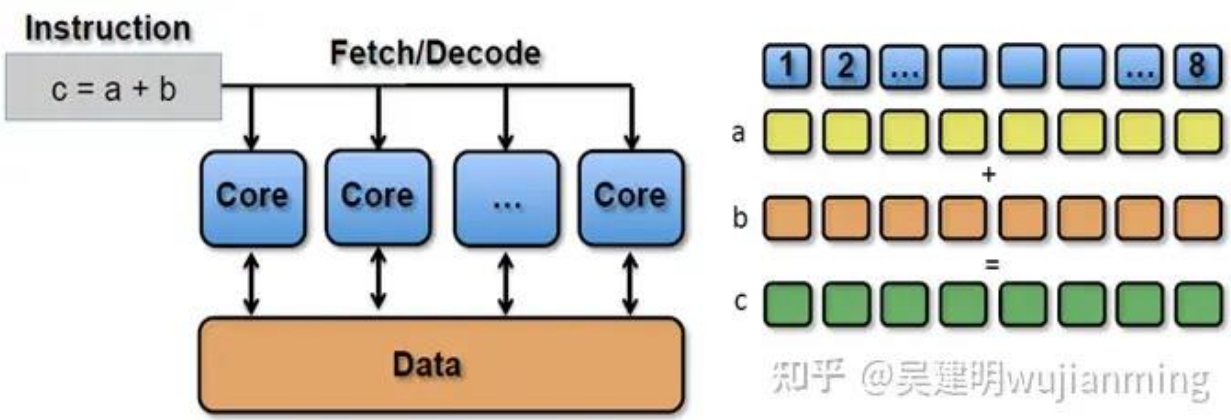
CSDN @杨咩咩ing

GPU中的SIMT

- 单指令多线程SIMT
  - GPU中指令并行技术SIMT ，是SIMD的升级版
  - 单条指令可以被多个线程同时执行。指令在不同线程中同时被执行时，其中存取、操作的数据可以是不同的。
  - 在使用时，会以SM为单位，SM单元是指令执行的“最小完整单元”。
  - 单个SM中的多个Core同时处理同一指令，每个core上运行一个线程，并且每个Core存取的数据可以是不同的。

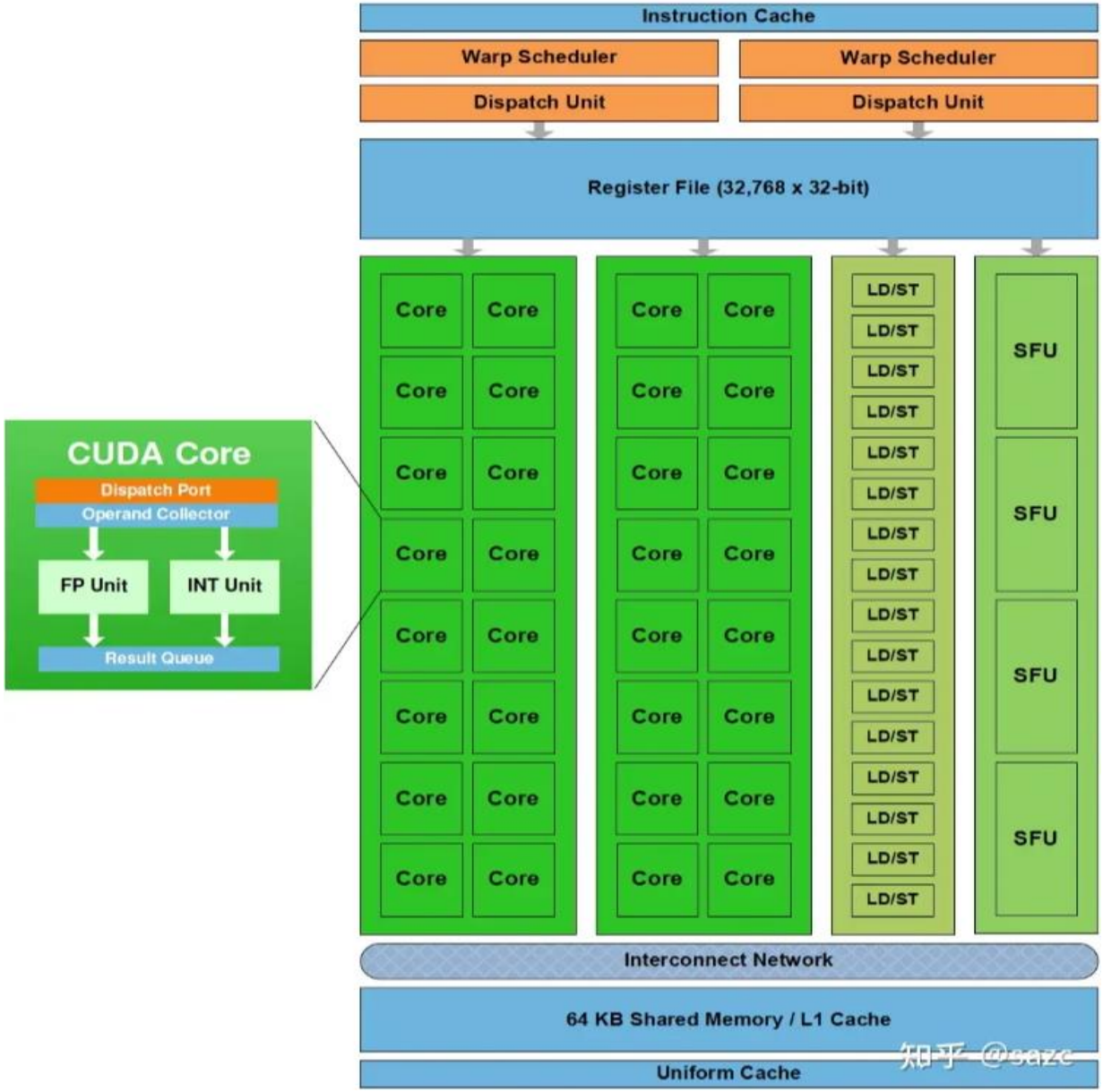
GPU中的SIMT

- **SIMT_ADD c, a, b**
  - **指令会被同时送入在单个SM中被编组的所有Core中，同时执行运算，但a、b 、c的值可以不一样**



GPU中的线程束

- 线程束
  - 线程束的大小，即一个线程束中线程的数量，由硬件的计算能力版本决定。在目前所有的**NVIDIA GPU**中，一个线程束由连续的**32**个线程组成。
  - 一个**SM**中有**32**个**core**，当一个线程束执行时，线程束中的**32**个线程会被分配到**32**个**core**中，每个**core**中单独运行一个线程，**32**个**core**运行相同的指令。
  - 线程束的好处就是，一个**warp**只需要一个套逻辑对指令进行解码和执行就可以了，芯片可以做的更小更快，之所以可以这么做是由于**GPU**需要处理的任务是天然并行的。


Instruction Cache
Warp Scheduler
Warp Scheduler
Dispatch Unit
Dispatch Unit
Register File (32,768 x 32-bit)
Core
LD/ST
SFU
CUDA Core
Dispatch Port
Operand Collector
FP Unit
INT Unit
Result Queue
Interconnect Network
64 KB Shared Memory / L1 Cache
Uniform Cache
知乎 @saze

GPU中的控制相关

- 线程束中的分支
  - 线程束内的线程共享同样的**PC**，以锁步的方式执行指令，但是每个线程又可以有自己的执行分支。虽然允许每个线程有不同的分支，但是当一个线程束中出现不同分支的线程时，会大大降低效率。
  - 如果一个**warp**中需要进行分支（即**warp**内的线程执行的指令指针指向不同的位置），性能将急剧的下降。如果一个**warp**内需要执行**N**个分支，那么**SM**就需要把每一个分支的指令发射到每一个**core**上，再由**core**根据线程的逻辑决定需不需要执行。这是一个串行过程，因此**SIMT**完成分支的时间是多个分支时间之和。

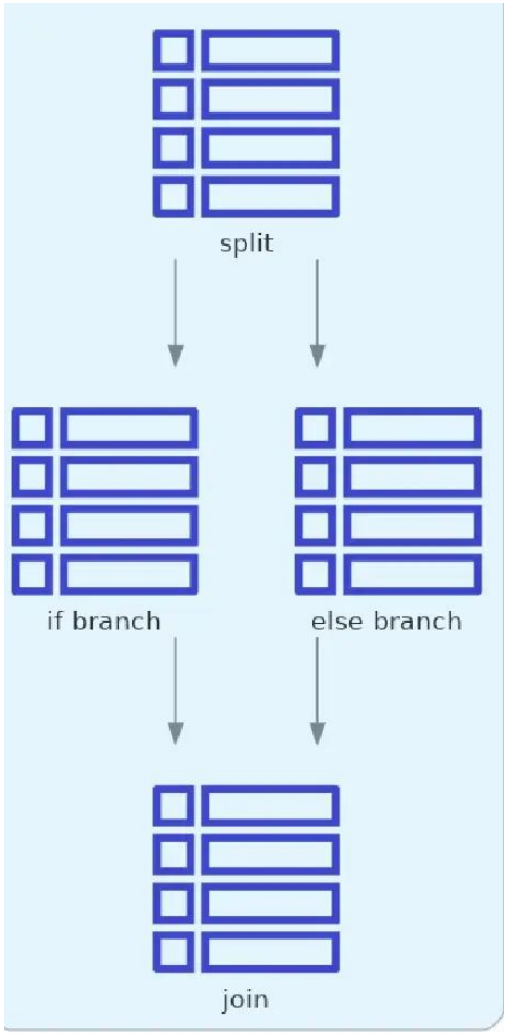
GPU中的并行计算

- 线程束中的分支
  - 假设一个**Warp**中有**16**个线程判断条件为真，另外**16**个线程条件为假，所以一半线程会执行**if**中的语句，另一半线程执行**else**中的语句。
  - 由于线程束中的线程同一时刻只能执行相同的指令。所以实际上遇到分支分歧时**GPU**会顺序执行每个分支路径，而禁用不在此路径上的线程，直到所有有线程使能的分支路径都走完，线程再重新汇合到同一执行路径。每个分支都有些线程空转，**Warp**实际上需要执行的指令数目大增。
  - 假设每个分支任务量大致相同，分支分歧造成的性能损失少则原先的一半，最坏的情况如果每个线程执行分支都不一致，性能下降为最高时候的**1/32**。

GPU中的并行计算

- 线程束中的分支
  - 假设一个**Warp**中有**16**个线程判断条件为真，另外**16**个线程条件为假，所以一半线程会执行**if**中的语句，另一半线程执行**else**中的语句。
  - 由于线程束中的线程同一时刻只能执行相同的指令。所以实际上遇到分支分歧时**GPU**会顺序执行每个分支路径，而禁用不在此路径上的线程，直到所有有线程使能的分支路径都走完，线程再重新汇合到同一执行路径。每个分支都有些线程空转，**Warp**实际上需要执行的指令数目大增。
  - 假设每个分支任务量大致相同，分支分歧造成的性能损失少则原先的一半，最坏的情况如果每个线程执行分支都不一致，性能下降为最高时候的**1/32**。

- 线程束中的分支
  - **NVIDIA使用了SIMT Stack的**方式来处理线程束中的分支问题
  - 在硬件层面增加两条指令：**split join**，用于在并行计算中实现线程束的分割和合并操作。
  - 这些指令能够将线程束分为更小的子线程束，并在之后将它们合并成更大的线程束。在执行分支时，另一个分支的信息需要保存，因此引入了**SIMT Stack**。



branch example

本章小结

1. 流水线的基本概念和分类
   - 流水技术的特点和流水线定义
   - 五种分类
2. 流水线性能的主要指标
   - 吞吐率、加速比、效率
   - 实际值和最大值
3. 流水线冲突
   - 结构冲突（瓶颈段细分、重复设置瓶颈段）
   - 数据冲突（分类、定向、暂停）
   - 控制冲突（分支改进、分支延迟）
4. 向量计算机
   - 向量数据表示
   - 向量流水线

第三章作业

王志英教材

第111页： T2、T3、T4、T5。

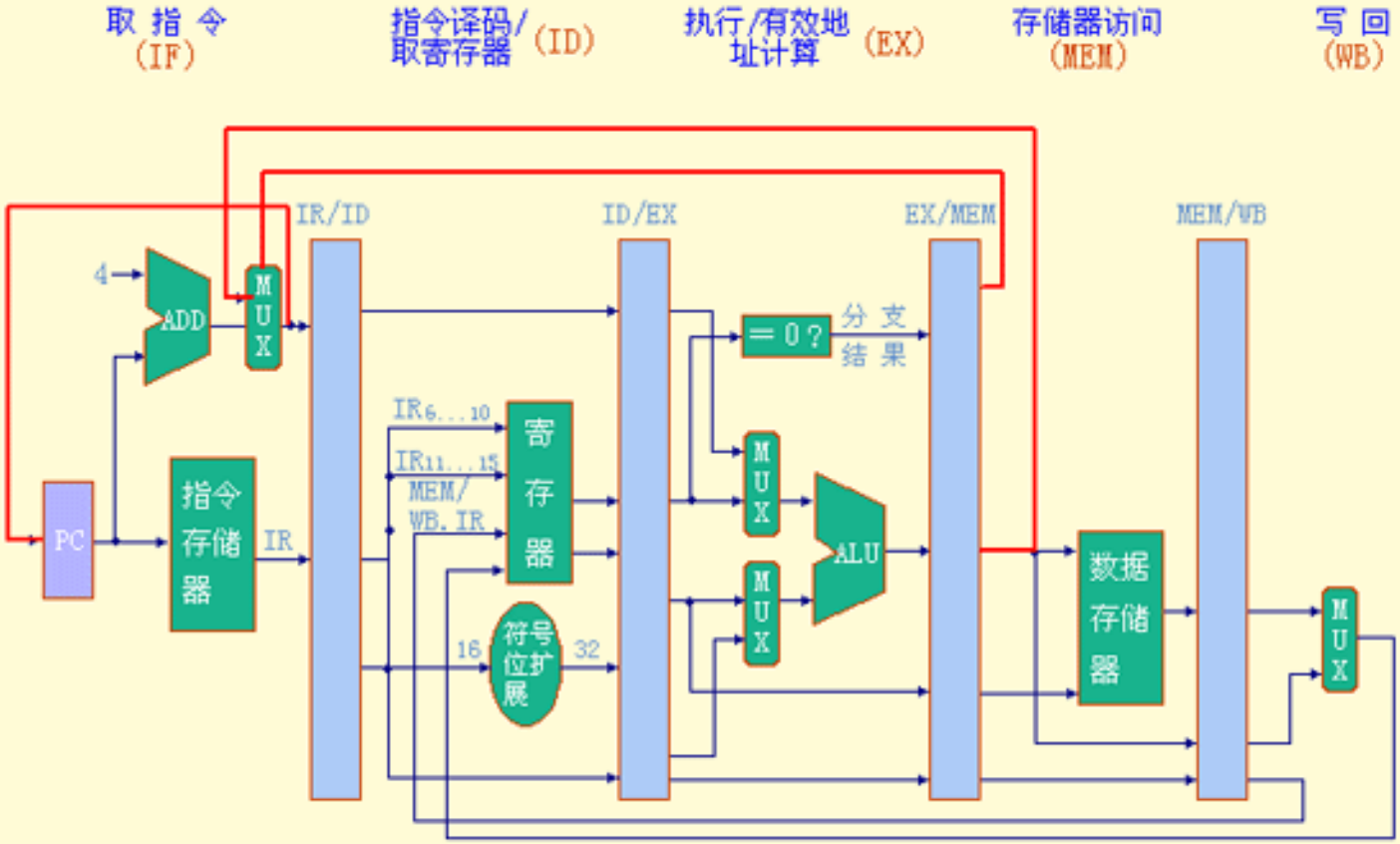
和第四章作业一起交。

课程报告

以下报告内容均需要引用本课程的相关内容：

- 选择你认为计算机发展历史上最重要的十台机器，并说明理由。
- 由于AI的广泛应用，CPU+GPU的架构越来越普遍，比较各厂家架构设计上的特点。
- 近五年来，在CPU、存储、总线、IO 等方面有什么重要的新进展，分析它的影响和未来的发展。

转移成功时，PC值的改变

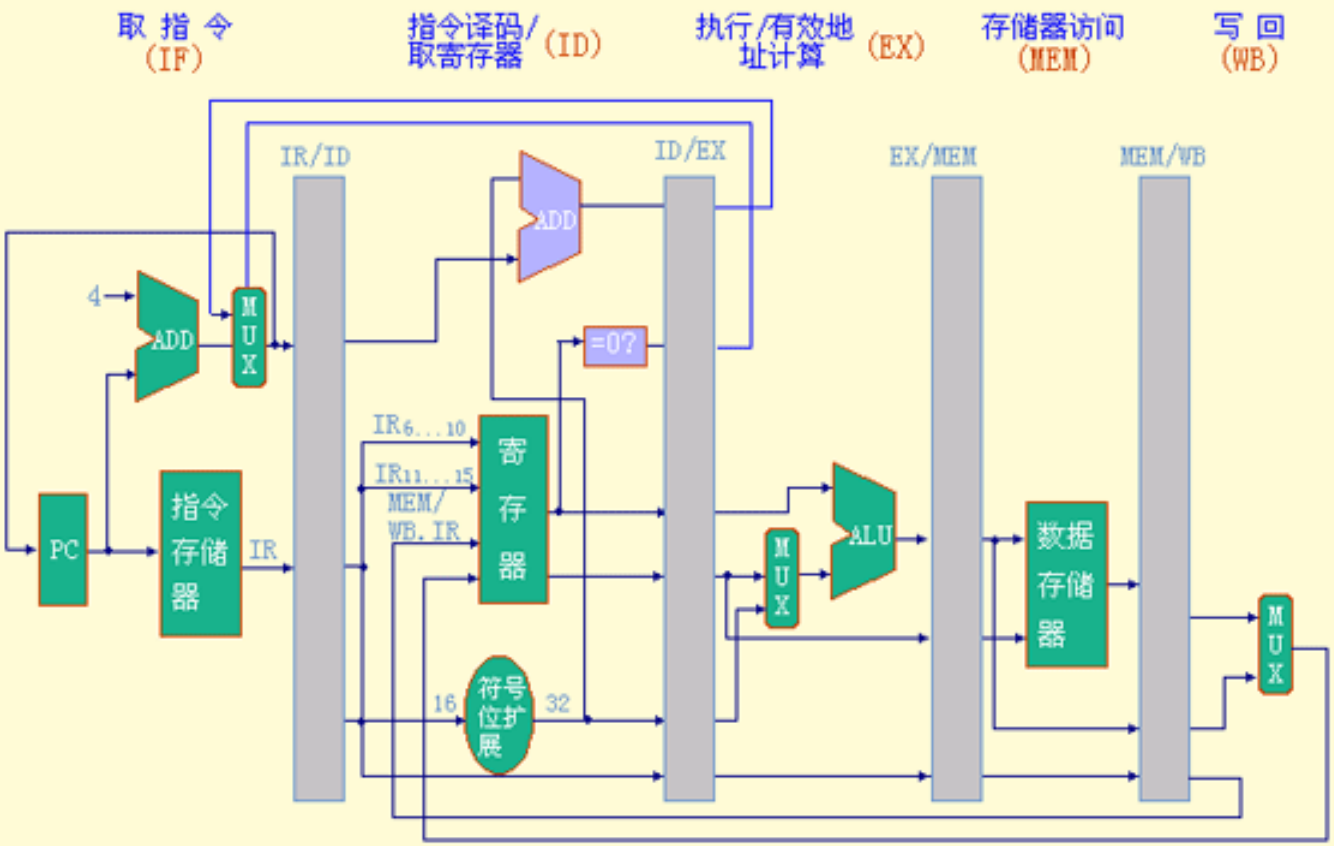


分支转移成功导致暂停 3 个时钟周期

(DLX流水线)

分支指令	IF	ID	EX	MEM	WB				
分支后继指令		IF	stall	stall	IF	ID	EX	MEM	WB
分支后继指令+1						IF	ID	EX	MEM	WB
分支后继指令+2							IF	ID	EX	MEM
分支后继指令+3								IF	ID	EX
分支后继指令+4									IF	ID
分支后继指令+5										IF

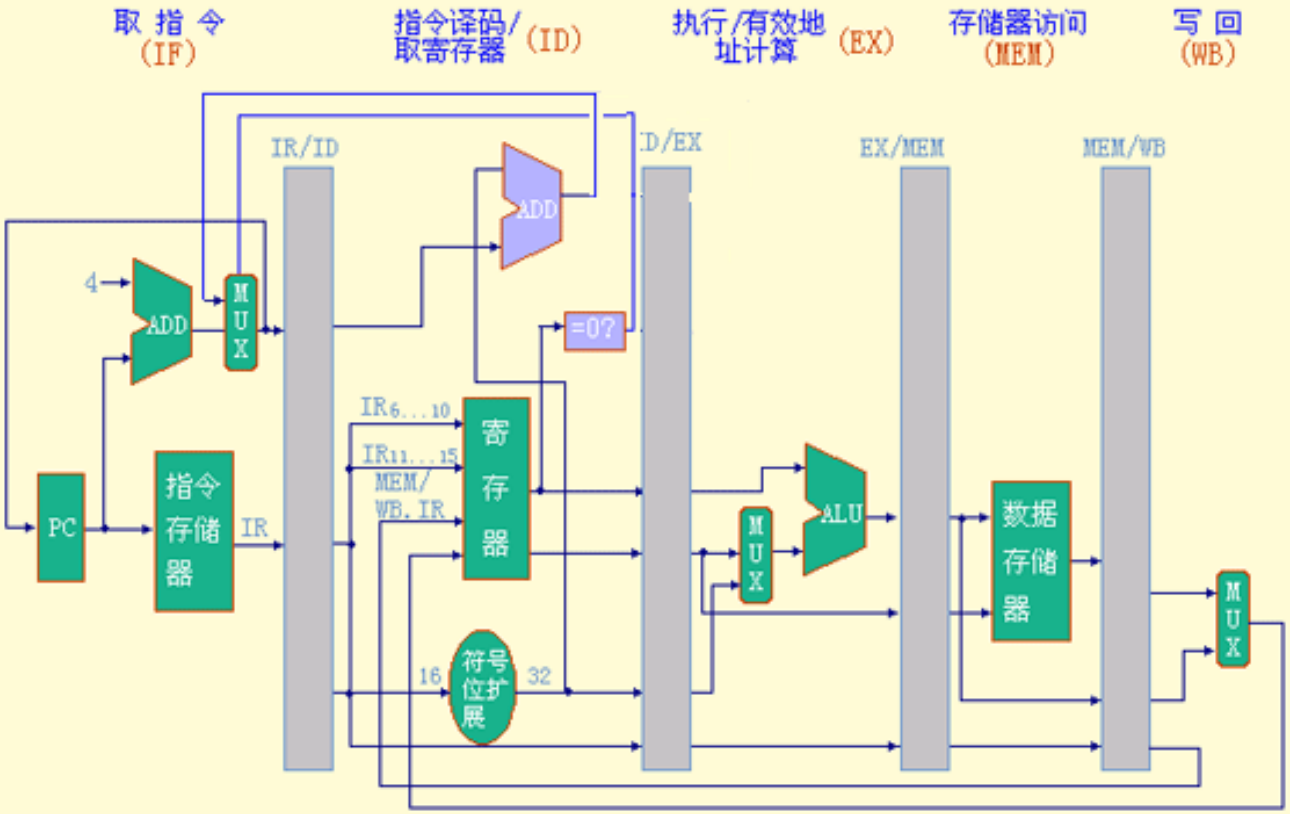
修改 DLX 以减少分支开销



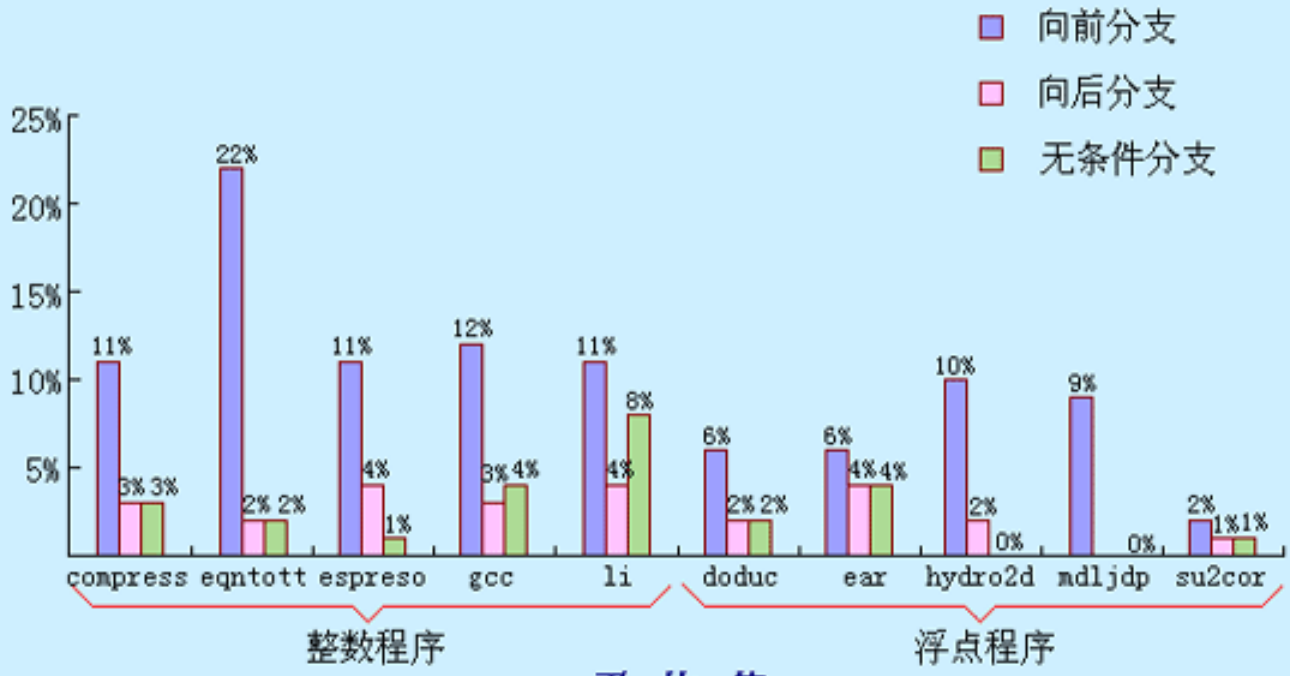
改进后流水线的分支操作

流 水 段	分 支 指 令 操 作
IF	IF/ID.IR ← Mem[PC]; IF/ID.NPC, PC ← (if ID/EX.Cond {ID/EX.NPC} else {PC+4});
ID	ID/EX.A ←Regs[IF/ID.IR $_{6..10}$]; ID/EX.B← Regs[IF/ID.IR $_{11..15}$]; ID/EX.IR ← IF/ID.IR; ID/EX.Imm ← (IF/ID.IR$_{16}$) ^{16} ## IF/ID.IR$_{16..31}$; ID/EX.cond ← (Regs[IF/ID.IR $_{6..10}$] op 0); ID/EX.NPC ← IF/ID.NPC +(IF/ID.IR$_{16}$)^{16} ## IF/ID.IR$_{16..31}$;
EX	
MEM	
WB	

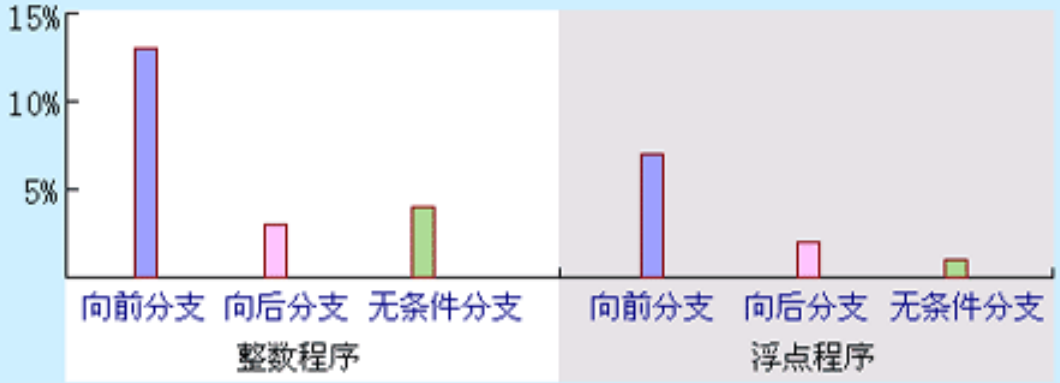
修改 DLX 以减少分支开销



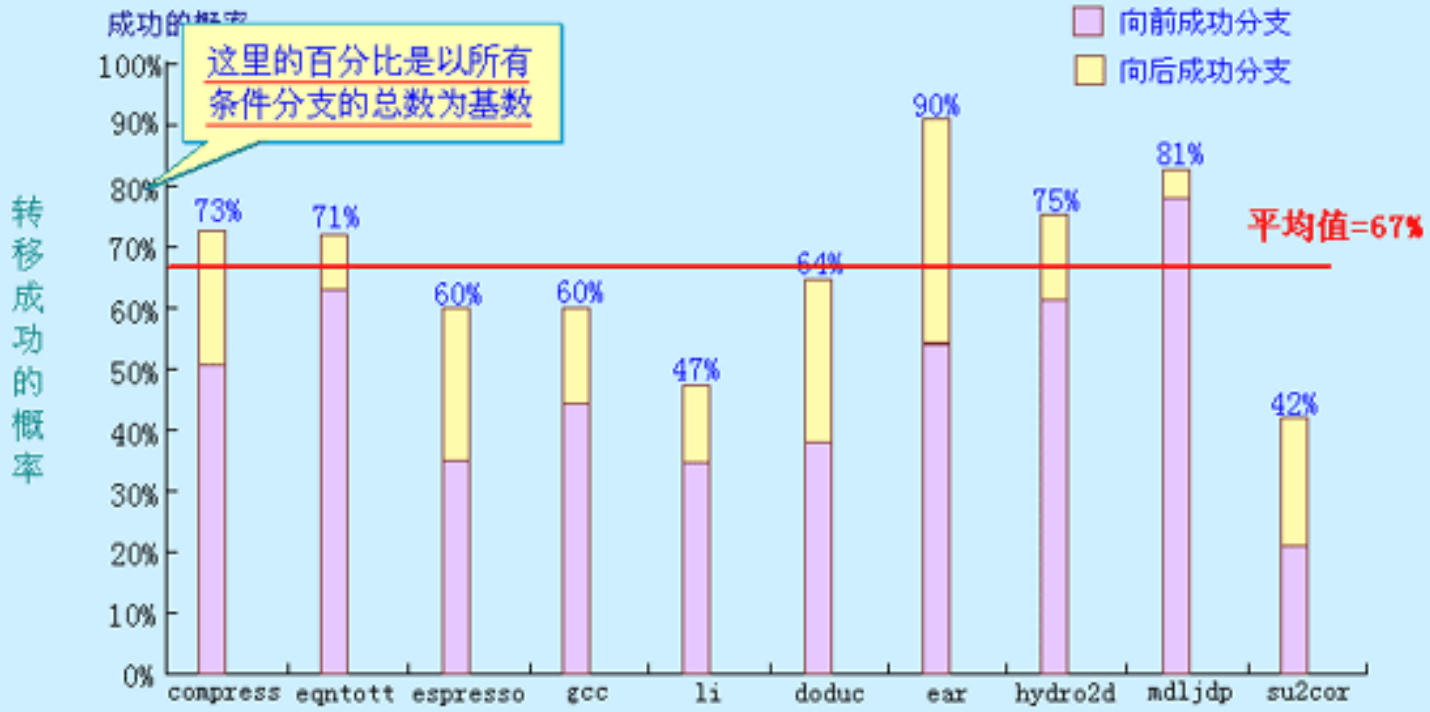
各种能改变PC值的指令的执行频率



平 均 值



条件分支转移成功的概率

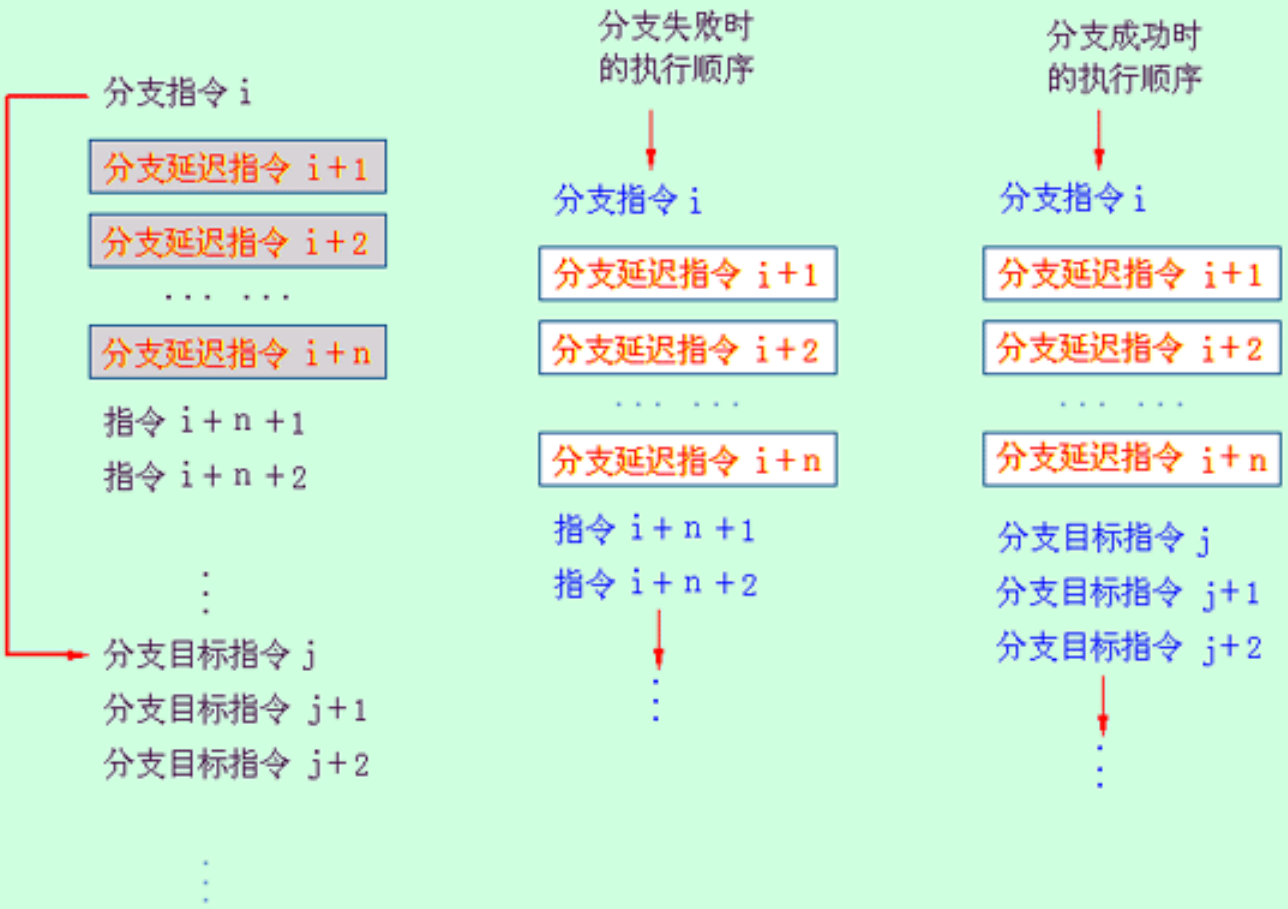


DLX流水线对分支的处理过程

分支指令 i(失败)	IF	ID	EX	MEM	WB				
指令 i+1		IF	ID	EX	MEM	WB			
指令 i+2			IF	ID	EX	MEM	WB		
指令 i+3				IF	ID	EX	MEM	WB	
指令 i+4					IF	ID	EX	MEM	WB

分支指令 i(成功)	IF	ID	EX	MEM	WB				
指令 i+1		IF	ID	idle	idle	idle			
分支目标 j			IF	ID	EX	MEM	WB		
分支目标 j+1				IF	ID	EX	MEM	WB	
分支目标 j+2					IF	ID	EX	MEM	WB

延迟分支以及指令的执行顺序



DLX流水线的执行过程

（具有一个分支延迟槽）

分支失败的情况

分支指令(失败)	IF	ID	EX	MEM	WB				
分支延迟指令(i+1)		IF	ID	EX	MEM	WB			
指令 i+2			IF	ID	EX	MEM	WB		
指令 i+3				IF	ID	EX	MEM	WB	
指令 i+4					IF	ID	EX	MEM	WB

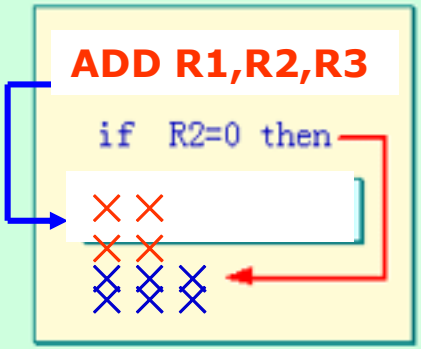
分支成功的情况

分支指令(成功)	IF	ID	EX	MEM	WB				
分支延迟指令(i+1)		IF	ID	EX	MEM	WB			
分支目标指令 j			IF	ID	EX	MEM	WB		
分支目标指令 j+1				IF	ID	EX	MEM	WB	
分支目标指令 j+2					IF	ID	EX	MEM	WB

分支延迟指令的三种调度方法

从前调度

调度后



Notake

If

ADD

ADD

if

take

If

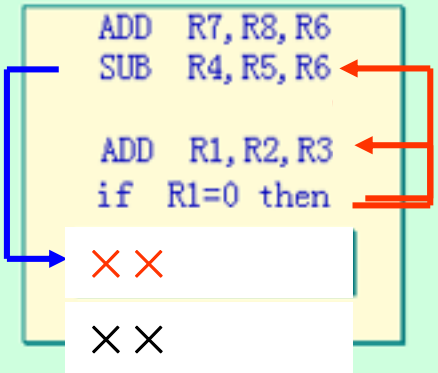
ADD

ADD

if

从目标处调度

调度后



Notake

ADD

SUB

ADD

if

SUB

Take

ADD

SUB

ADD

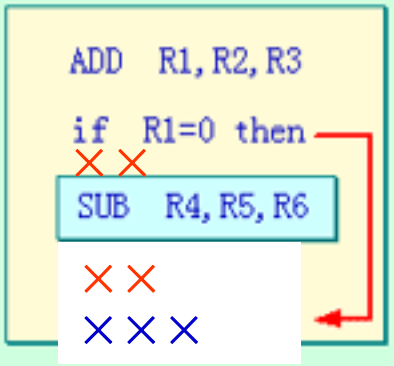
if

SUB

ADD

从失败处调度

调度后



Notake

ADD

If

SUB

SUB

Take

ADD

If

SUB

三种方法的要求及效果

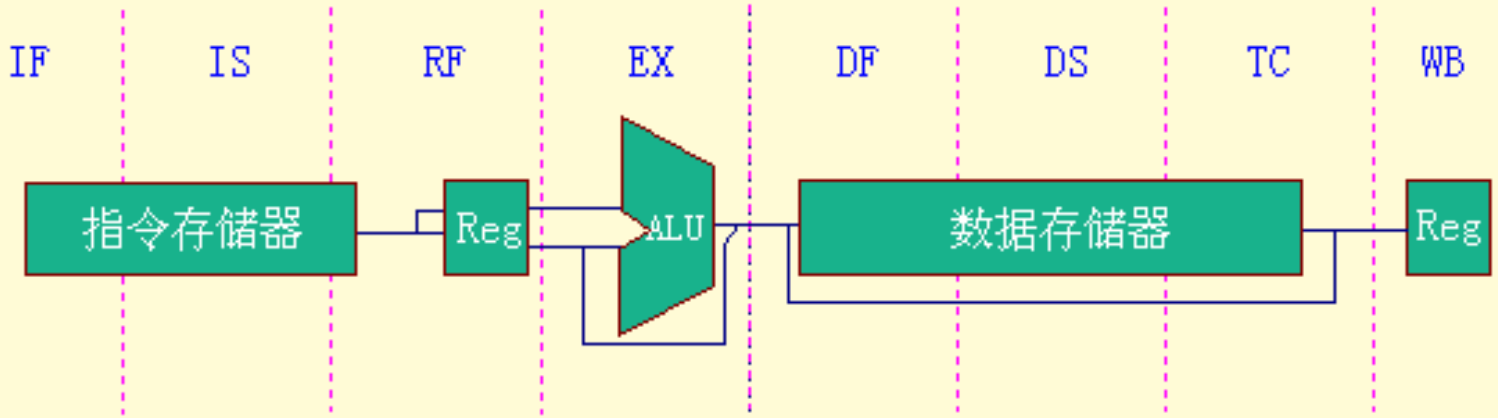
调度策略	对调度的要求	其作用前提
从前调度	被调度的指令必须与分支结果无关	任何情况
从目标处调度	必须保证在分支失败时执行被调度的指令不会导致错误，可能需要复制指令	分支成功时
从失败处调度	必须保证在分支成功时执行被调度的指令不会导致错误	分支失败时

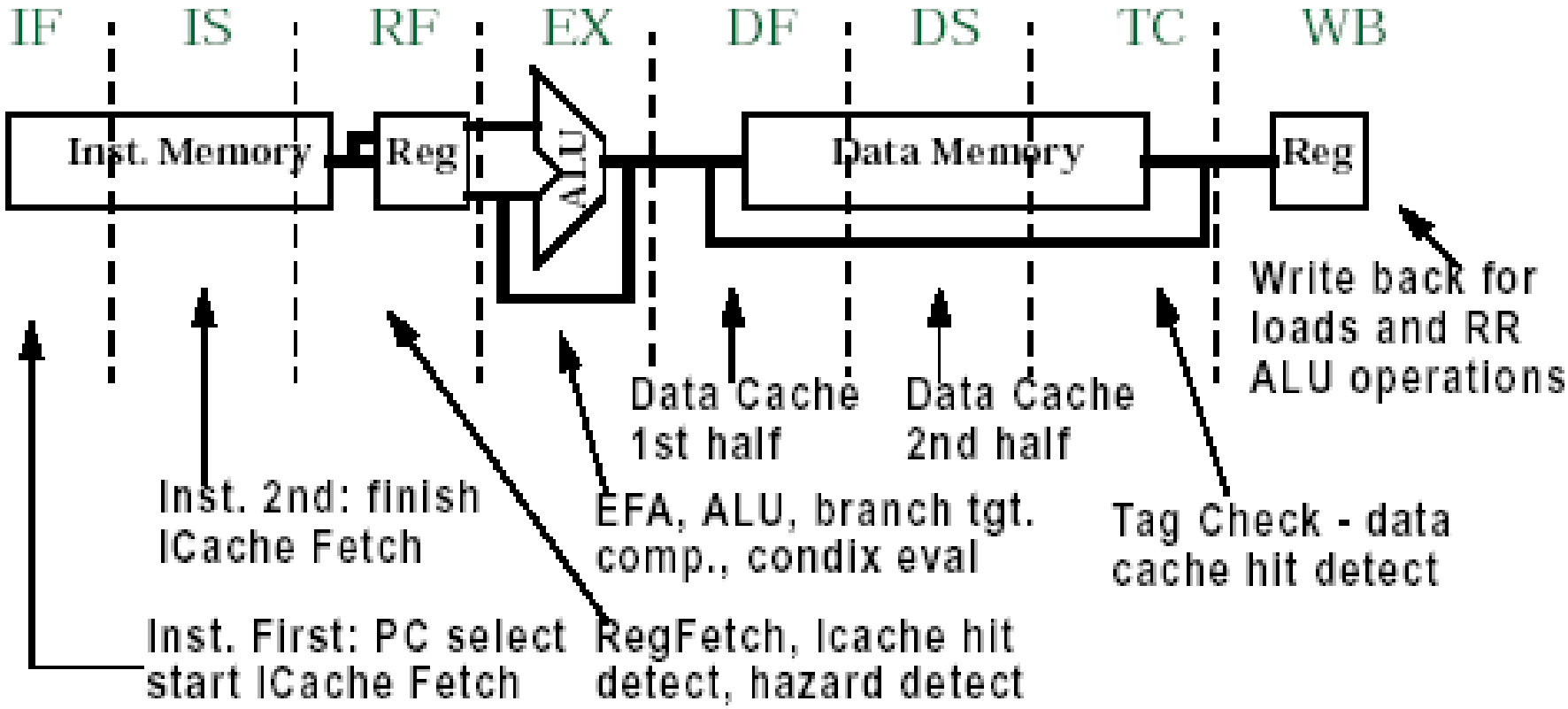
“预测成功--取消”分支的执行过程

分支指令 i(成功)	IF	ID	EX	MEM	WB				
分支延迟指令 i+1		IF	ID	EX	MEM	WB			
分支目标 j			IF	ID	EX	MEM	WB		
分支目标 j+1				IF	ID	EX	MEM	WB	
分支目标 j+2					IF	ID	EX	MEM	WB

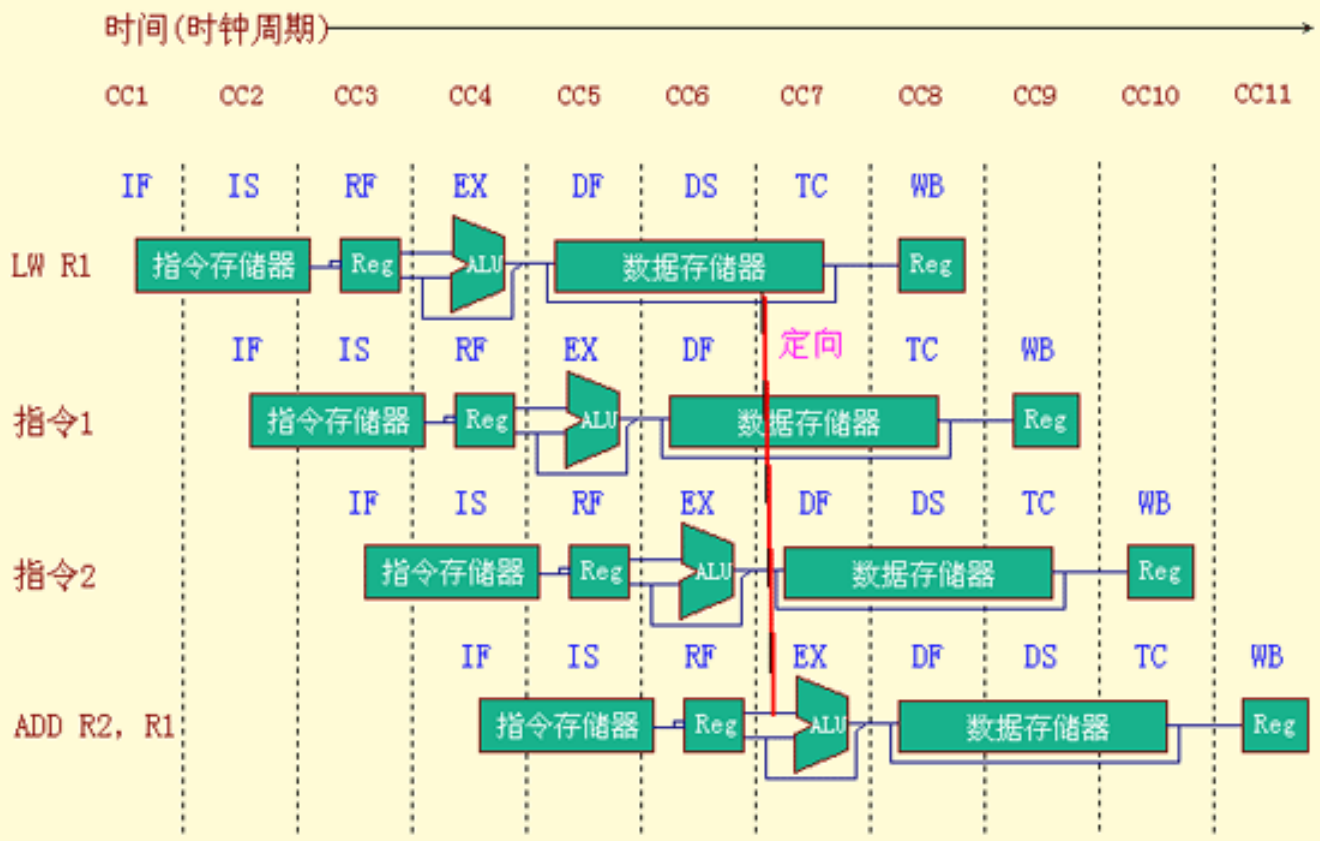
分支指令 i(失败)	IF	ID	EX	MEM	WB				
分支延迟指令 i+1		IF	ID	idle	idle	idle			
指令 i+2			IF	ID	EX	MEM	WB		
指令 i+3				IF	ID	EX	MEM	WB	
指令 i+4					IF	ID	EX	MEM	WB

R4000流水线的结构

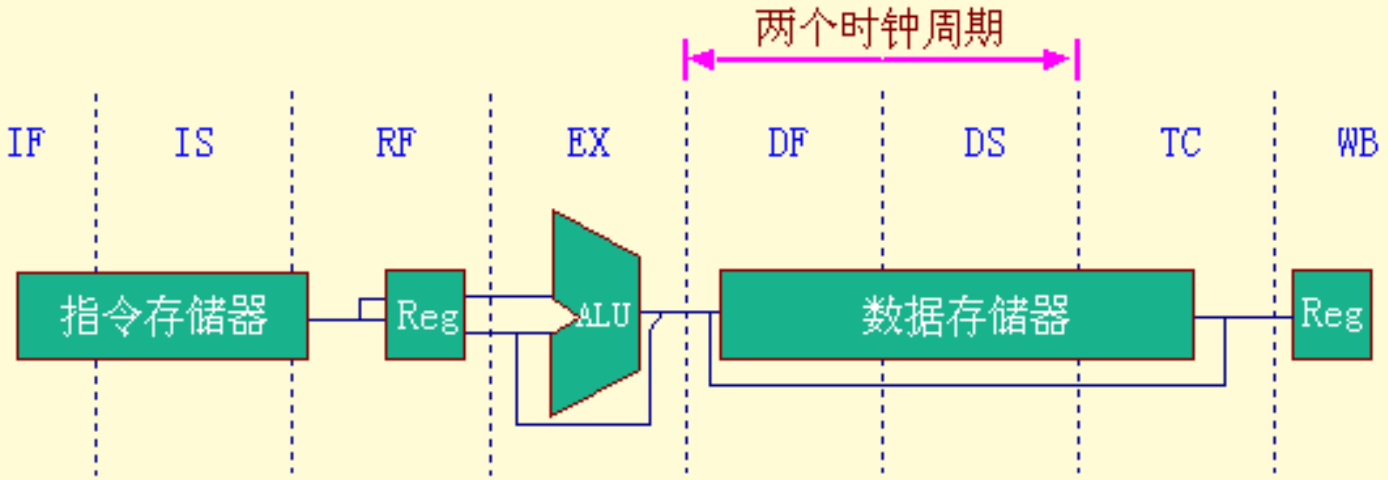



IF
IS
RF
EX
DF
DS
TC
WB
Inst. Memory
Reg
ALU
Data Memory
Reg
Write back for loads and RR ALU operations
Data Cache 1st half
Data Cache 2nd half
Inst. 2nd: finish ICache Fetch
EFA, ALU, branch tgt. comp., condix eval
Tag Check - data cache hit detect
Inst. First: PC select start ICache Fetch
RegFetch, Icache hit detect, hazard detect

指令序列在R4000流水线中的重叠执行过程



载入延迟为两个时钟周期

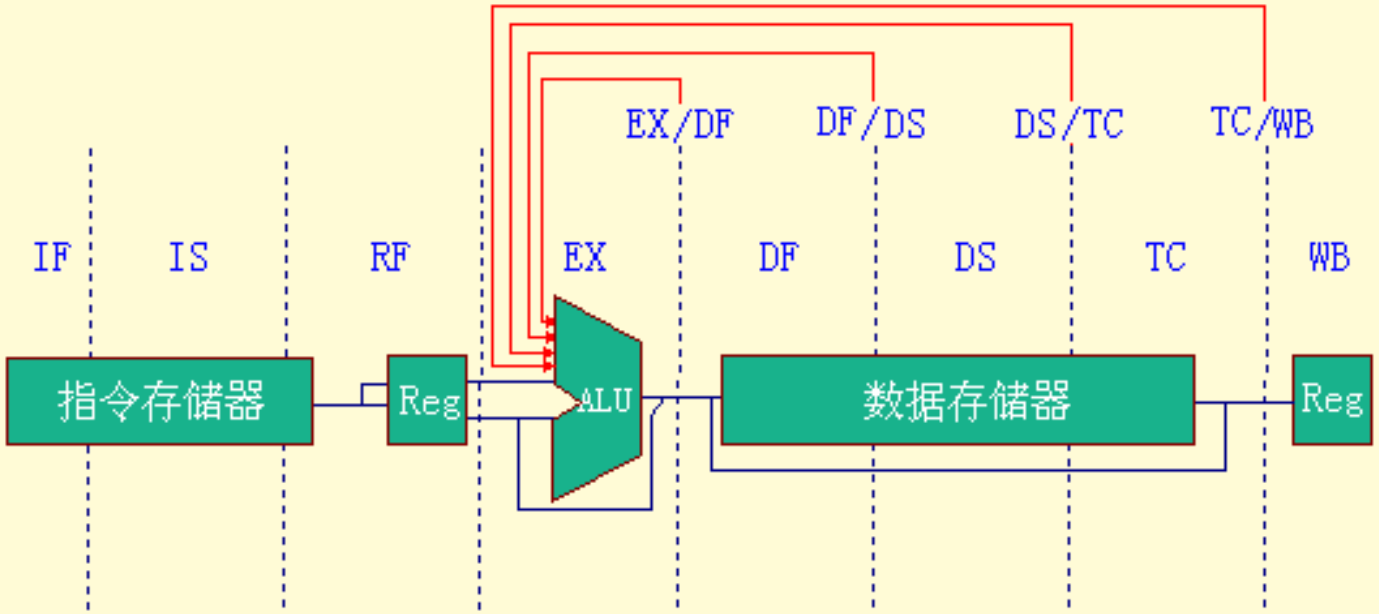


指令系列在R4000流水线中的执行时空图

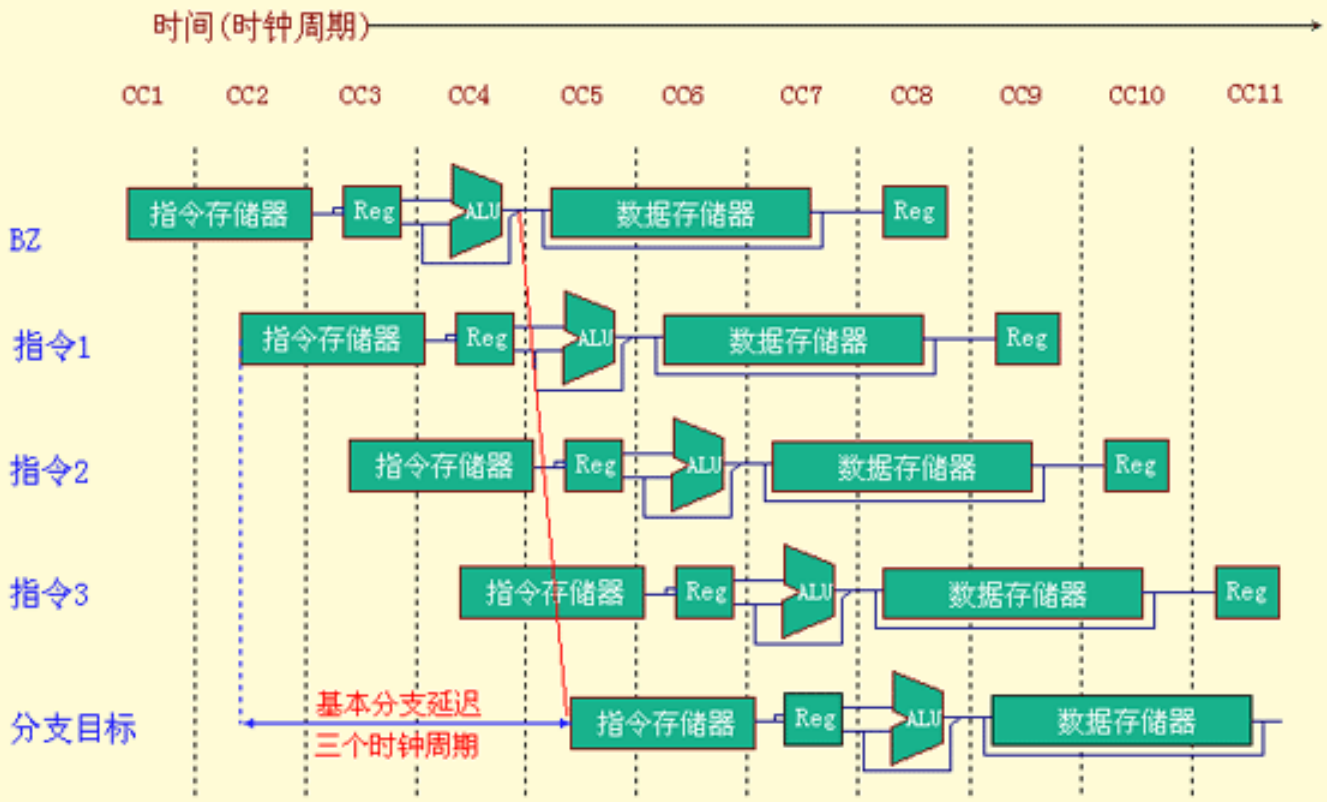
指令序列	时钟周期								
	1	2	3	4	5	6	7	8	9
LW R1		IS	RF	EX	DF	DS	TC	WB	
ADD R2, R1			IS	RF	stall	stall	EX	DF	DS
SUB R3, R1			IF	IS	stall	stall	RF	EX	DF
OR R4, R1				IF	stall	stall	IS	RF	EX

定向

R4000的流水线中ALU输入有四个定向源



R4000流水线的基本分支延迟



R4000流水线处理分支指令的时空图

指令序列	时钟周期								
	1	2	3	4	5	6	7	8	9
分支指令	IF	IS	RF	EX	DF	DS	TC	WB	
延迟槽		IF	IS	RF	EX	DF	DS	TC	WB
暂停			stall	stall	stall	stall	stall	stall	stall
暂停				stall	stall	stall	stall	stall	stall
分支目标					IF	IS	RF	EX	DF

两个暂停周期

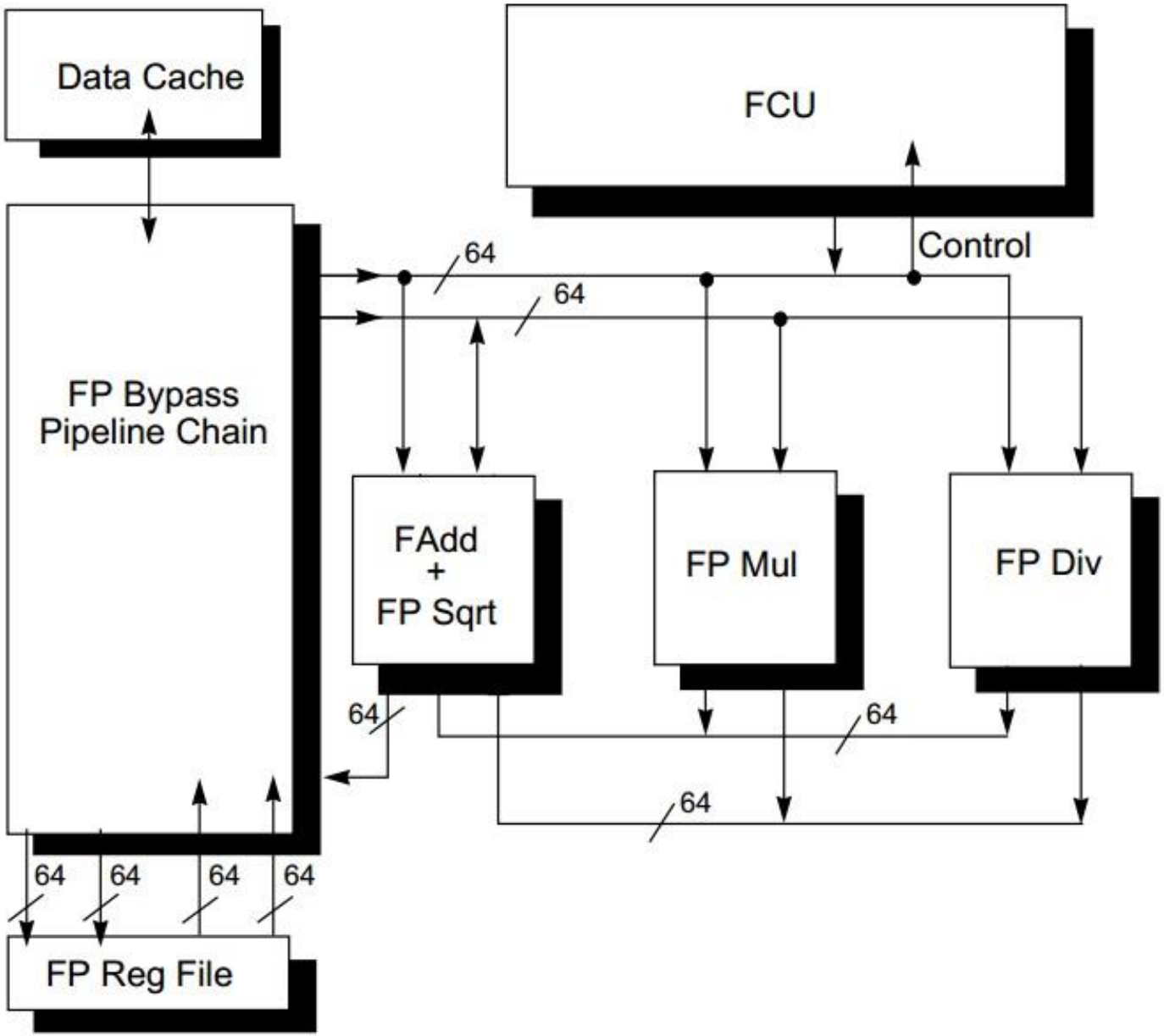


Figure 6-1 FPU Functional Block Diagram

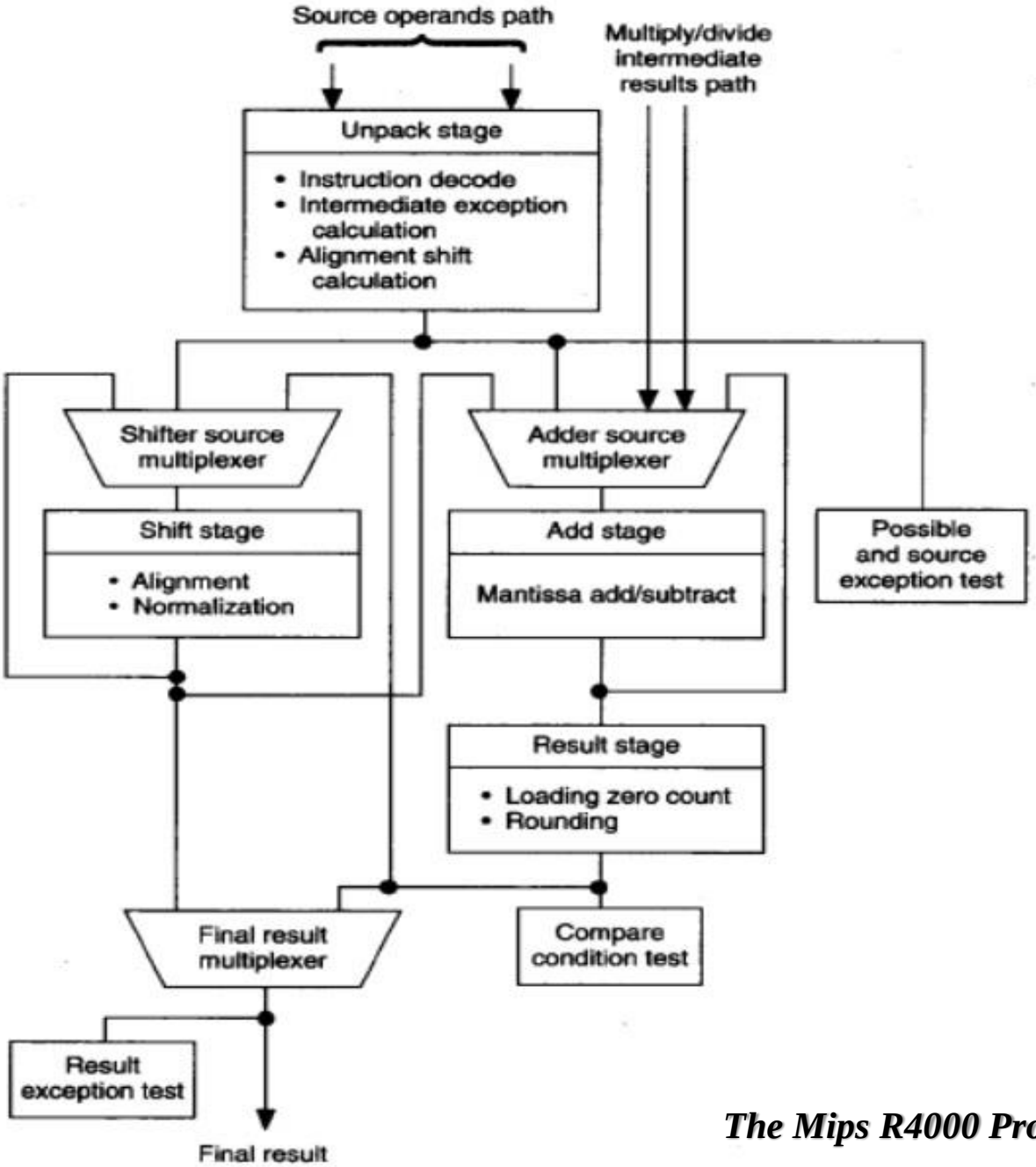
mips_r4000_users_manual

R4000浮点流水线中8个流水段

流水段	功能部件	描述
A	浮点加法器	尾数加
D	浮点除法器	除法
E	浮点乘法器	例外测试
M	浮点乘法器	乘法第一阶段
N	浮点乘法器	乘法第二阶段
R	浮点加法器	舍入
S	浮点加法器	操作数移位
U		展开浮点数

双精度浮点操作指令延迟、初始化间隔和流水段的使用情况

浮点指令	延迟	初始化间隔	使用的流水段
加、减	4	3	U,S+A,A+R,R+S
乘	8	4	U,E+M,M,M,M,N,N+A,R
除	36	35	U,A,R,D^{28},D+A,D+R,D+A,D+R,A,R
求平方根	112	111	U,E,$(A+R)^{108}$,A,R
取反	2	1	U,S
求绝对值	2	1	U,S
浮点比较	3	2	U,A,R



The Mips R4000 Processor-IEEE Micro 1992

Figure 7. Adder logical block diagram.

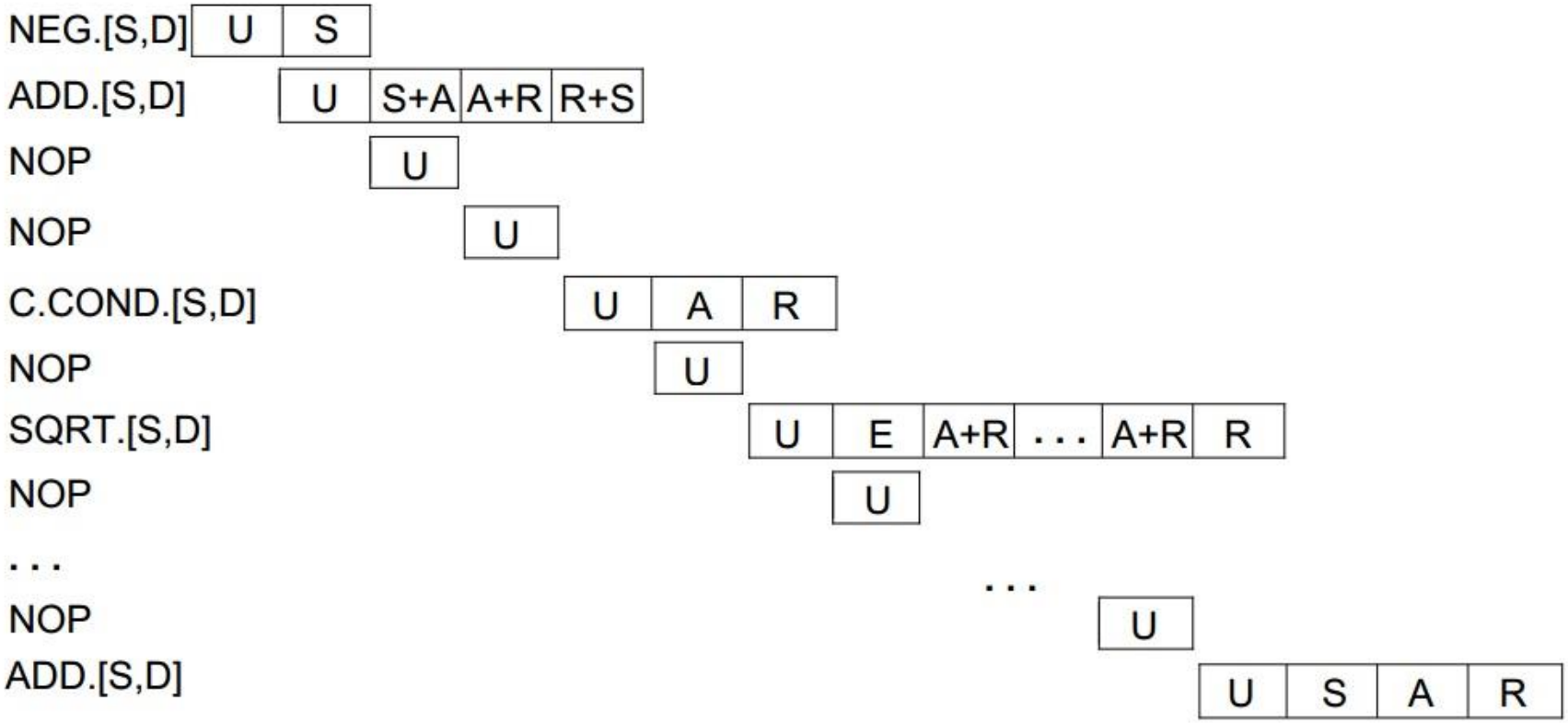


Figure 6-13 Instruction Cycle Overlap in FPU Adder

mips_r4000_users_manual

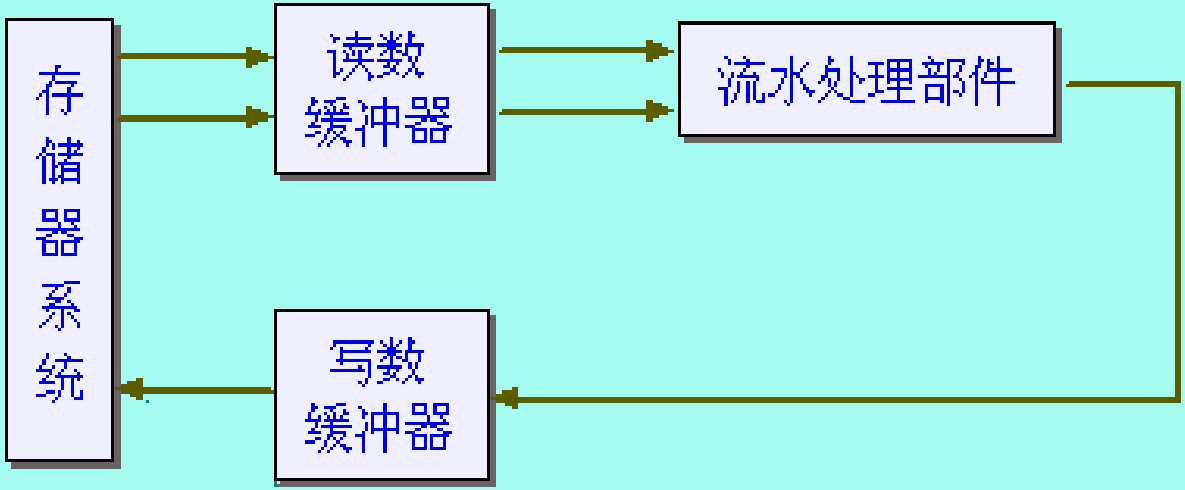
向量A、B、C、D

$$\mathbf{A} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_N \end{pmatrix} \qquad \mathbf{B} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{pmatrix} \qquad \mathbf{C} = \begin{pmatrix} C_1 \\ C_2 \\ \vdots \\ C_N \end{pmatrix} \qquad \mathbf{D} = \begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_N \end{pmatrix}$$

水平（横向）处理方式

$$\begin{pmatrix} a_1 * (b_1 + C_1) \\ a_1 * (b_2 + C_2) \\ \vdots \\ a_N * (b_N + C_N) \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_N \end{pmatrix} * \left(\begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{pmatrix} + \begin{pmatrix} C_1 \\ C_2 \\ \vdots \\ C_N \end{pmatrix} \right)$$

存储器--存储器型操作的运算流水线

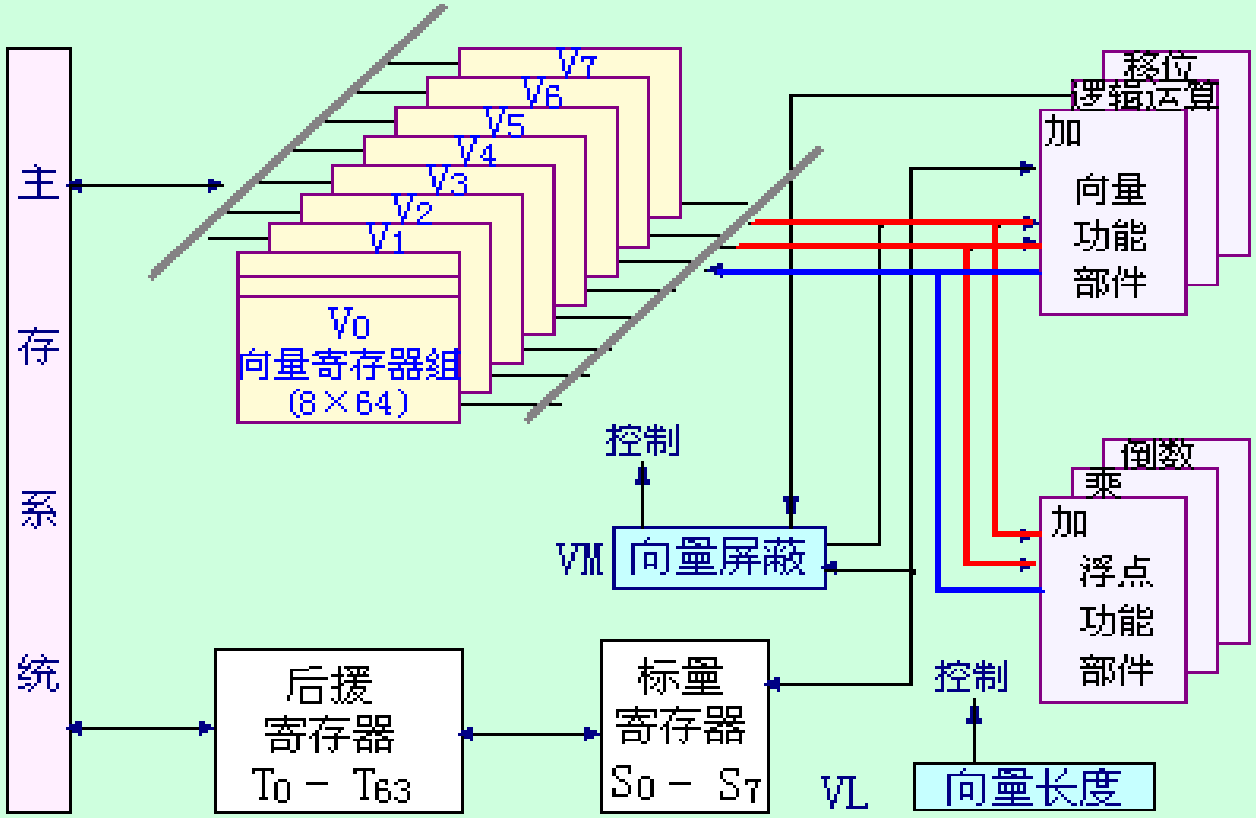


分组（纵横）处理方式

$$
\begin{array}{ccccccc}
D & & A & & B & & C \\
\begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_{n-1} \end{pmatrix} & & \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{pmatrix} & & \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_{n-1} \end{pmatrix} & & \begin{pmatrix} C_1 \\ C_2 \\ \vdots \\ C_{n-1} \end{pmatrix} \\
\hline
\begin{pmatrix} d_n \\ d_{n+1} \\ \vdots \\ d_{2n-1} \end{pmatrix} & = & \begin{pmatrix} a_n \\ a_{n+1} \\ \vdots \\ a_{2n-1} \end{pmatrix} & * \left(\right. & \begin{pmatrix} b_n \\ b_{n+1} \\ \vdots \\ b_{2n-1} \end{pmatrix} & + & \left. \begin{pmatrix} C_n \\ C_{n+1} \\ \vdots \\ C_{2n-1} \end{pmatrix} \right) \\
\hline
\begin{pmatrix} d_{2n} \\ d_{2n+1} \\ \vdots \\ d_{3n-1} \end{pmatrix} & & \begin{pmatrix} a_{2n} \\ a_{2n+1} \\ \vdots \\ a_{3n-1} \end{pmatrix} & & \begin{pmatrix} b_{2n} \\ b_{2n+1} \\ \vdots \\ b_{3n-1} \end{pmatrix} & & \begin{pmatrix} C_{2n} \\ C_{2n+1} \\ \vdots \\ C_{3n-1} \end{pmatrix} \\
\hline
\begin{pmatrix} \vdots \end{pmatrix} & & \begin{pmatrix} \vdots \end{pmatrix} & & \begin{pmatrix} \vdots \end{pmatrix} & & \begin{pmatrix} \vdots \end{pmatrix}
\end{array}
$$

$$
\begin{array}{ccccccc}
D & & A & & B & & C \\
\begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_{n-1} \end{pmatrix} & = & \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{pmatrix} & * \left(\right. & \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_{n-1} \end{pmatrix} & + & \left. \begin{pmatrix} C_1 \\ C_2 \\ \vdots \\ C_{n-1} \end{pmatrix} \right) \\
\begin{pmatrix} d_n \\ d_{n+1} \\ \vdots \\ d_{2n-1} \end{pmatrix} & = & \begin{pmatrix} a_n \\ a_{n+1} \\ \vdots \\ a_{2n-1} \end{pmatrix} & * \left(\right. & \begin{pmatrix} b_n \\ b_{n+1} \\ \vdots \\ b_{2n-1} \end{pmatrix} & + & \left. \begin{pmatrix} C_n \\ C_{n+1} \\ \vdots \\ C_{2n-1} \end{pmatrix} \right) \\
\begin{pmatrix} d_{2n} \\ d_{2n+1} \\ \vdots \\ d_{3n-1} \end{pmatrix} & = & \begin{pmatrix} a_{2n} \\ a_{2n+1} \\ \vdots \\ a_{3n-1} \end{pmatrix} & * \left(\right. & \begin{pmatrix} b_{2n} \\ b_{2n+1} \\ \vdots \\ b_{3n-1} \end{pmatrix} & + & \left. \begin{pmatrix} C_{2n} \\ C_{2n+1} \\ \vdots \\ C_{3n-1} \end{pmatrix} \right)
\end{array}
$$

寄存器—寄存器型运算流水线



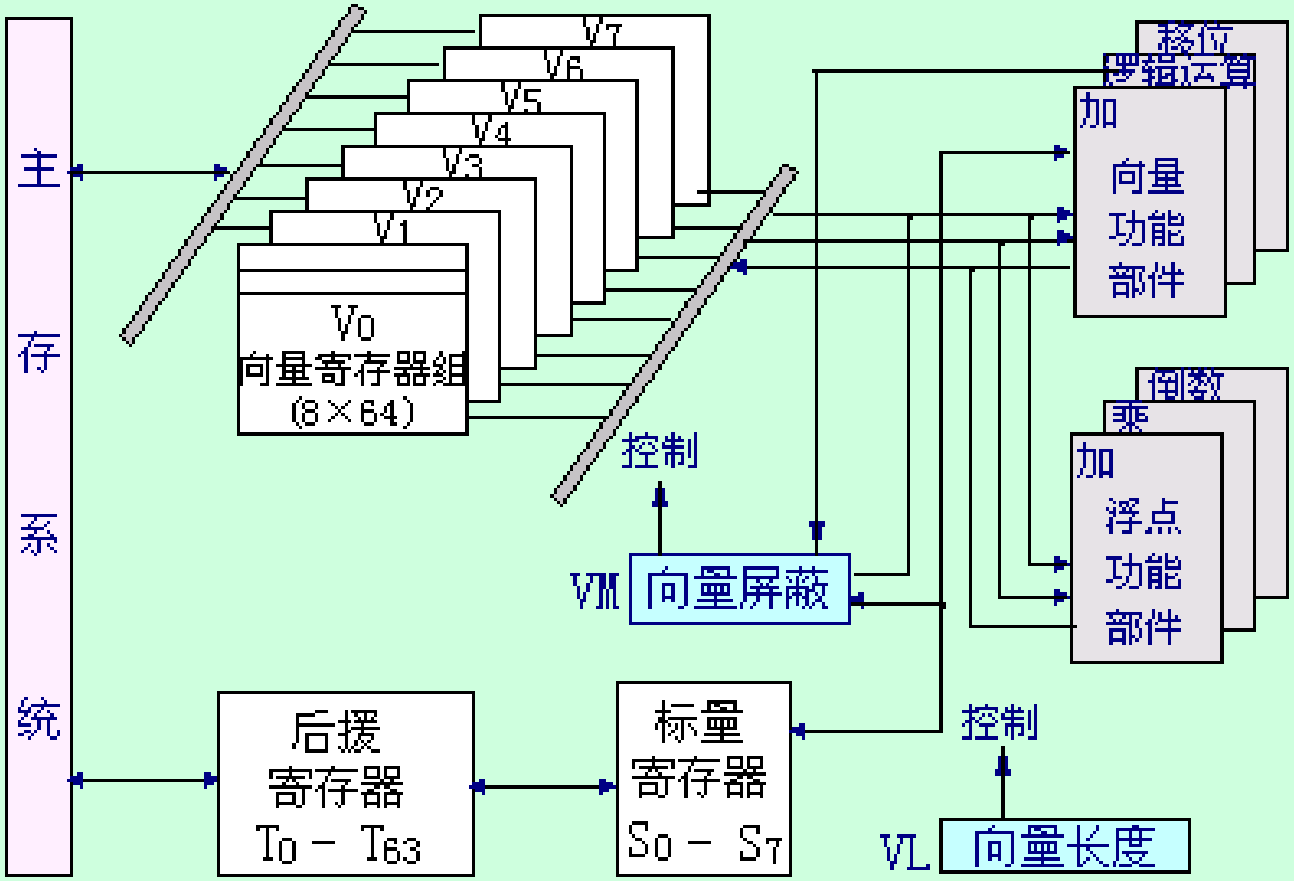
```
S1←5.0          ；将常数5.0送入标量寄存器S1
S2←1.0          ；将常数1.0送入标量寄存器S2
VL←N            ；在向量长度寄存器VL中设
                ；置向量长度N
V0←B            ；从存储器中将向量B读入向
                ；量寄存器V0
V1←S1*V0        ；向量B中的每个元素分别和
                ；常数S1相乘
V2←S2+V1        ；向量V1中的每个元素分别
                ；和常数S2相加
A←V2            ；将计算结果从向量寄存器
                ；V2存入存储器的向量A
```

假　设　N=K*64+L

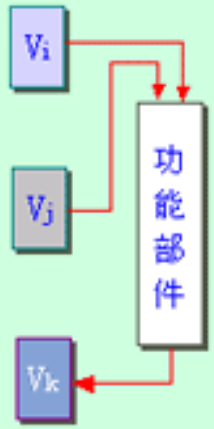
```
S1←5.0;
S2←1.0;
VL←L;
V0←B;
V1←S1*V0;
V2←S2+V1;
A←V2;
```

```
VL←64;
for (I=0 to K-1)  {
    V0←B;
    V1←S1*V0;
    V2←S2+V1;
    A←V2;
}
```

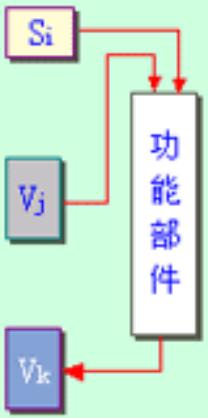
CRAY-1的基本结构


主
存
系
统
V7
V6
V5
V4
V3
V2
V1
V0
向量寄存器组
(8×64)
移位
逻辑运算
加
向量
功能
部件
倒数
乘
加
浮点
功能
部件
控制
VM
向量屏蔽
后援
寄存器
T0 – T63
标量
寄存器
S0 – S7
控制
VL
向量长度

CRAY-1的向量指令类型



$V_k \leftarrow V_i$ op V_j

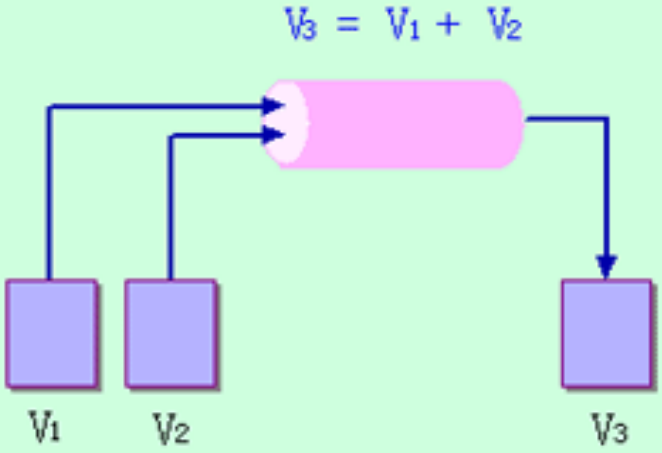


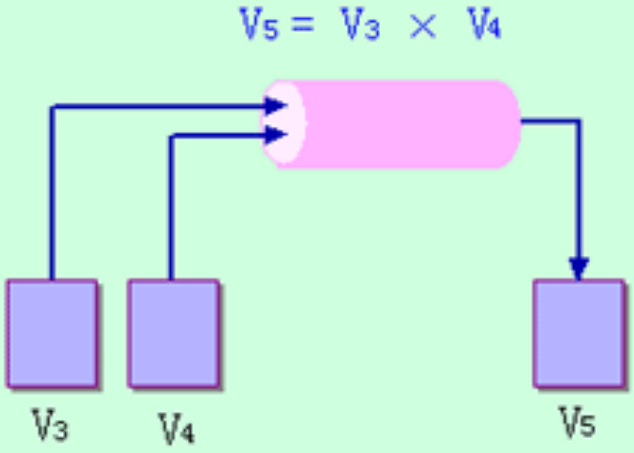
$V_k \leftarrow S_i$ op V_j

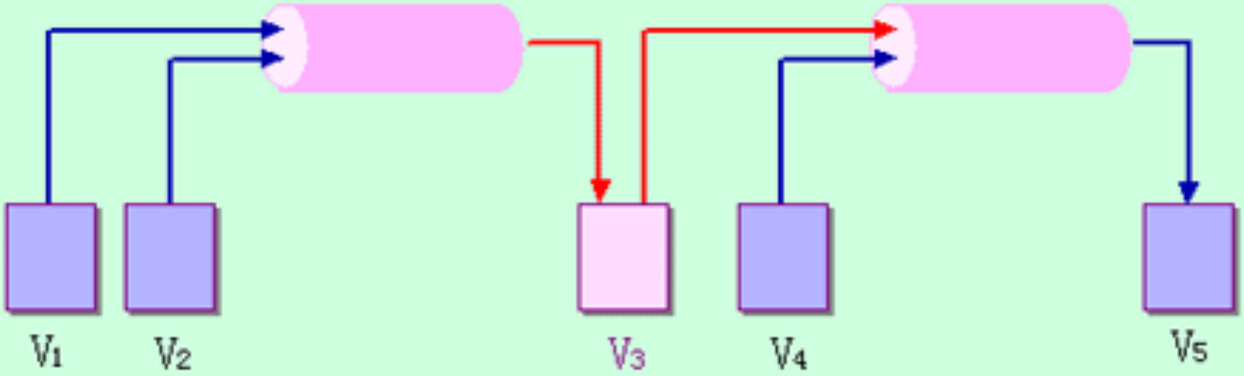
$V_k \leftarrow$ 主存

主存 $\leftarrow V_i$

链接特征


$V_3 = V_1 + V_2$
V_1
V_2
V_3


$V_5 = V_3 \times V_4$
V_3
V_4
V_5


$V_3 = V_1 + V_2$
$V_5 = V_3 \times V_4$
V_1
V_2
V_3
V_4
V_5